



UNIVERSIDAD PERUANA
CAYETANO HEREDIA

Neuronas Biológicas El perceptrón Multi Layer Perceptron (MLP) y Backpropagation

Msc. Lewis De La Cruz



cayetano.edu.pe



Hitos

LeNet: reconoce códigos postales, Yann Lecun, Bernhard Boser y otros, lo ejecutaron en vivo en el servicio postal de Estados Unidos.

Reconocimiento de Dígitos

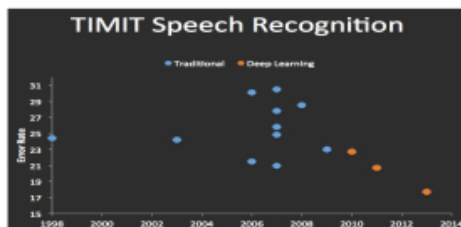
1989



1997

Reconocimiento de Voz

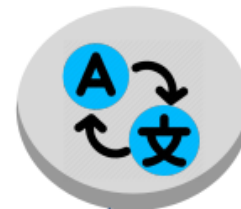
Recurrent Nets: LSTMs



Convolutional NNs: AlexNet: entrenada sobre 200GB de Datos de ImageNet

Clasificación de Imágenes

2012



2014

Traducción de Idiomas

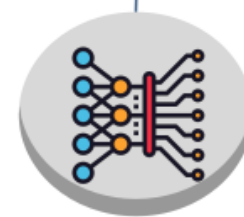
Modelos secuencia a secuencia con LSTM y atención:

2010-2012 - redes neuronales - speech-to-text y reconocimiento de objetos

2016 - AlphaGo derrotó al campeón humano de Go

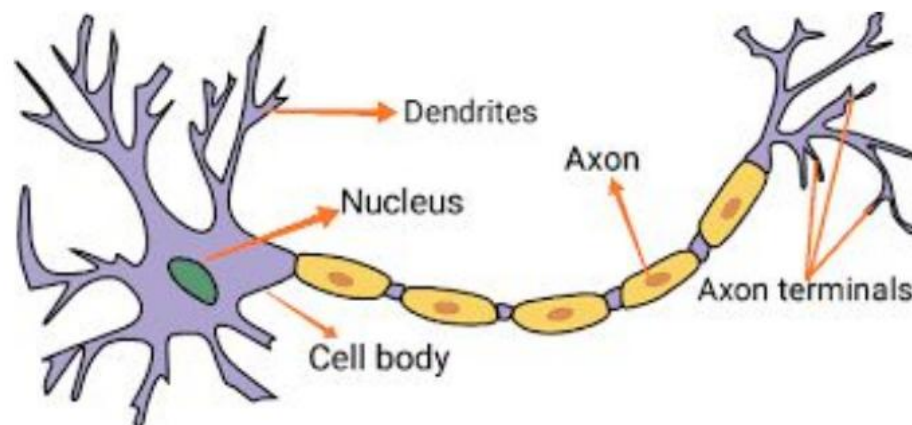
Aprendizaje Profundo

2016



Deep Learning

Redes neuronales biológicas



vs

Redes neuronales artificiales

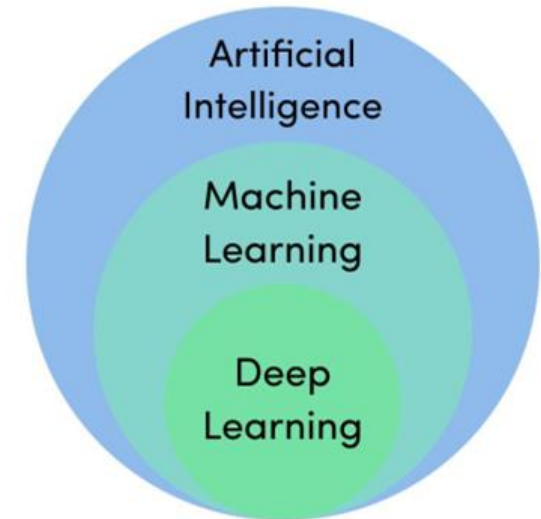


Redes neuronales biológicas:

- "Unidad computacional" elemental del cerebro.
- El cerebro tiene aproximadamente 85 mil millones neuronas.
- Las neuronas interactúan y activan señales eléctricas.
- Aprenden a cuando activar dadas ciertas entradas.

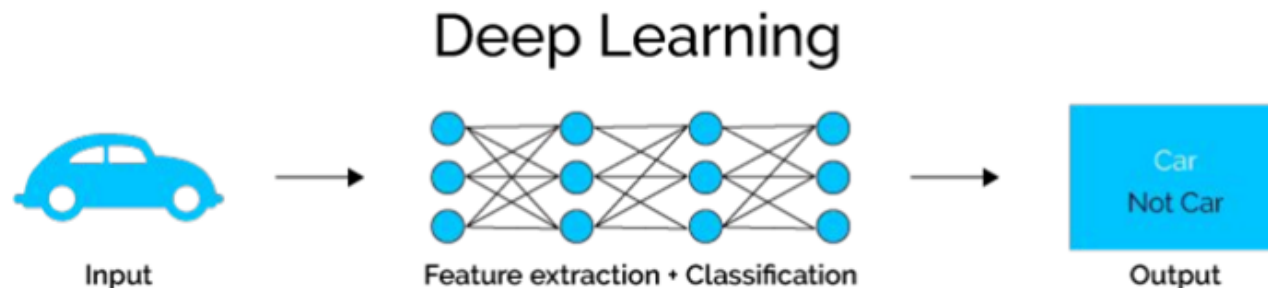
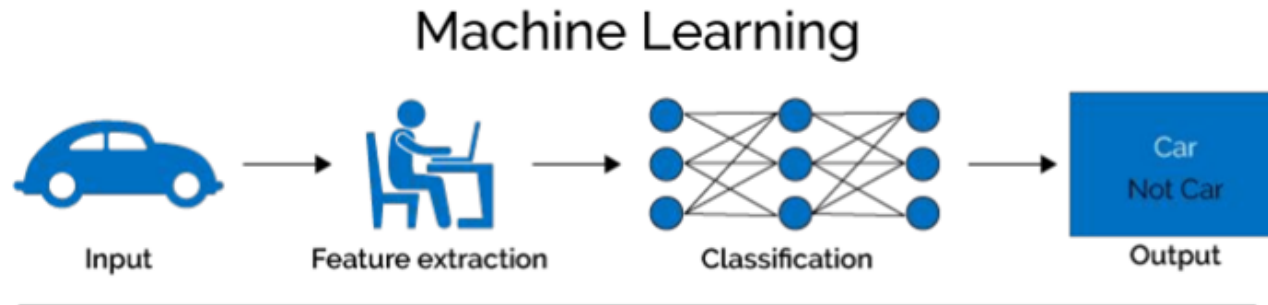
Deep Learning

¿Redes neuronales?



¿Qué es el deep learning?

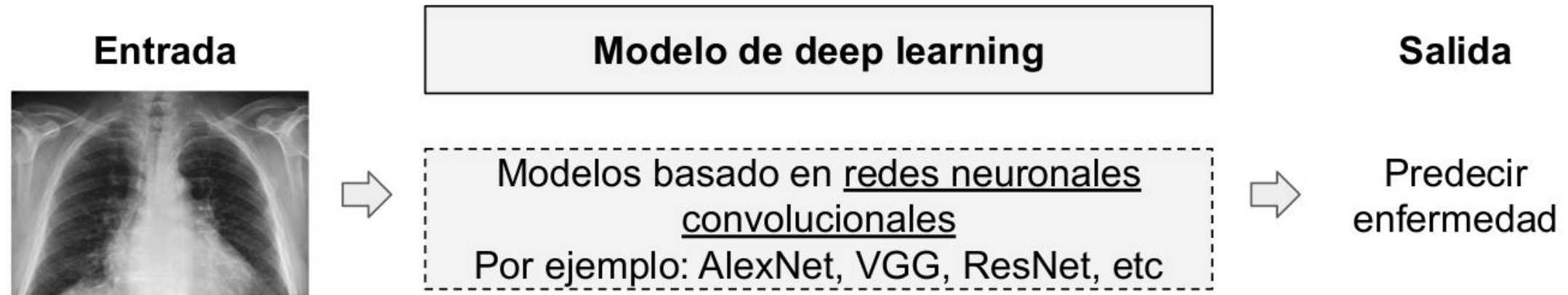
- Un subcampo de *machine learning* de aprendizaje de representaciones de datos.
- Excepcionalmente efectivo en aprendizaje de patrones.
- Los algoritmos de aprendizaje profundo intentan aprender (múltiples niveles de) representación mediante el uso de una jerarquía de múltiples capas
- Si le proporciona al sistema toneladas de información, comienza a comprenderla y a responder de manera útil.



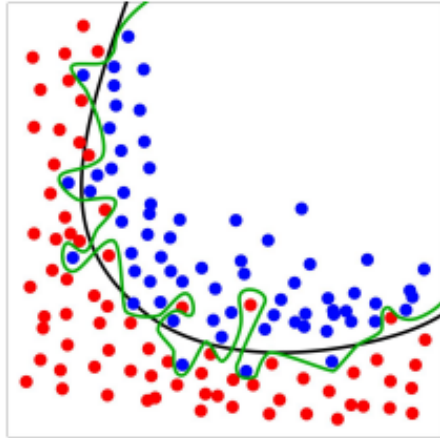
Modelos tradicionales de machine learning:



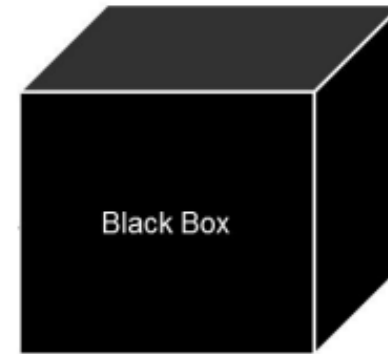
Deep learning



Problemas con Deep Learning

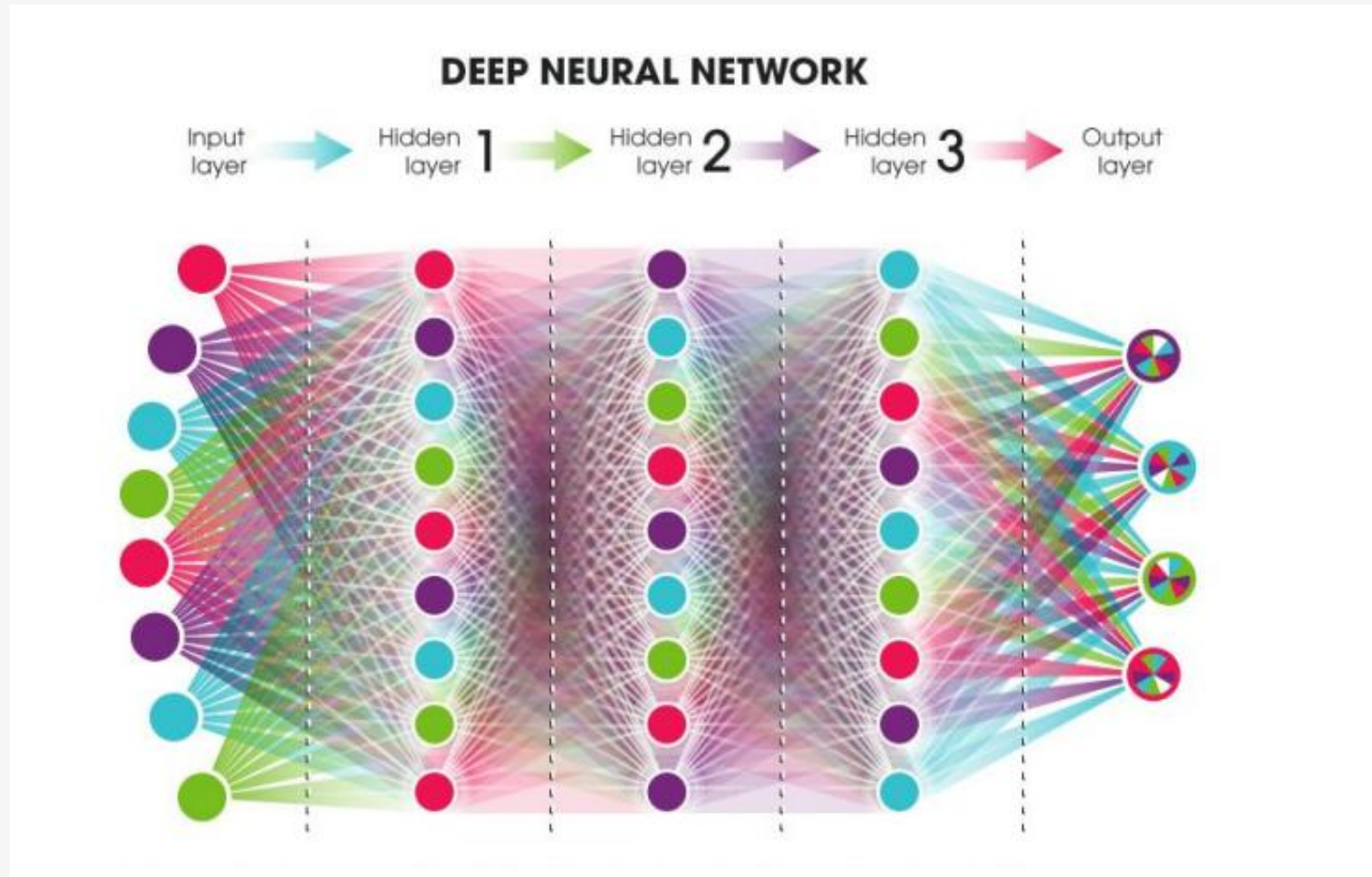


Overfitting
(Problemas de
sobre ajuste)

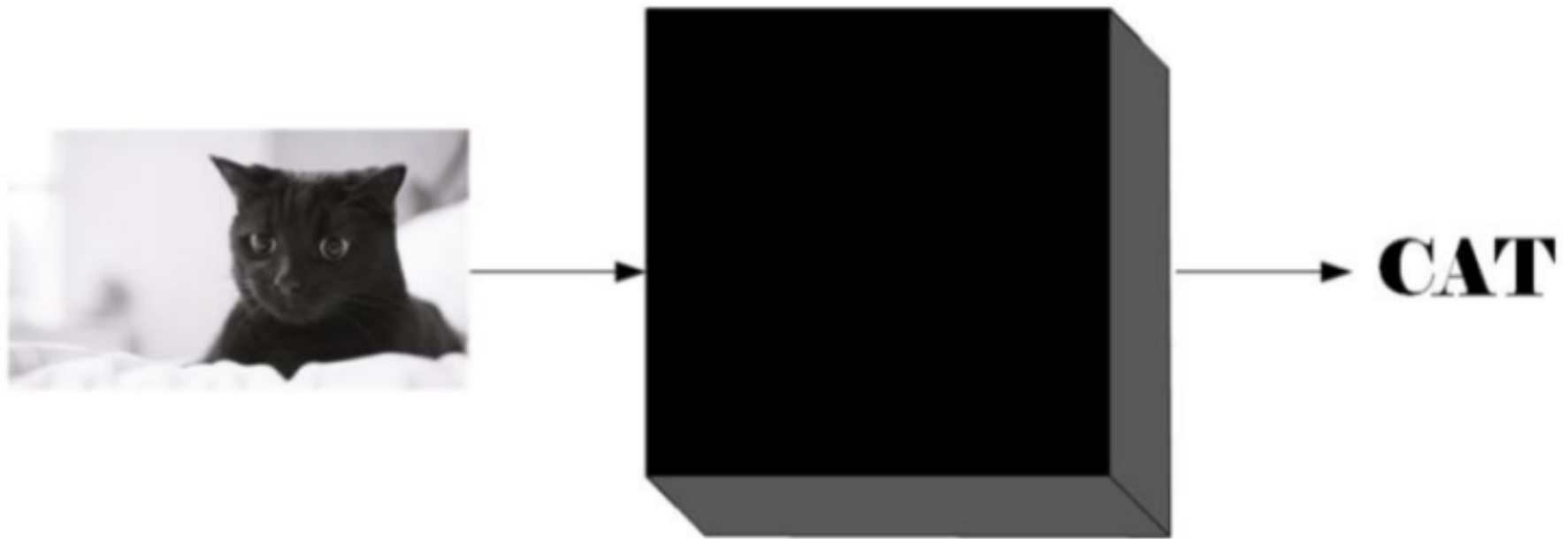


Caja negra
¿Qué está
ocurriendo
adentro?

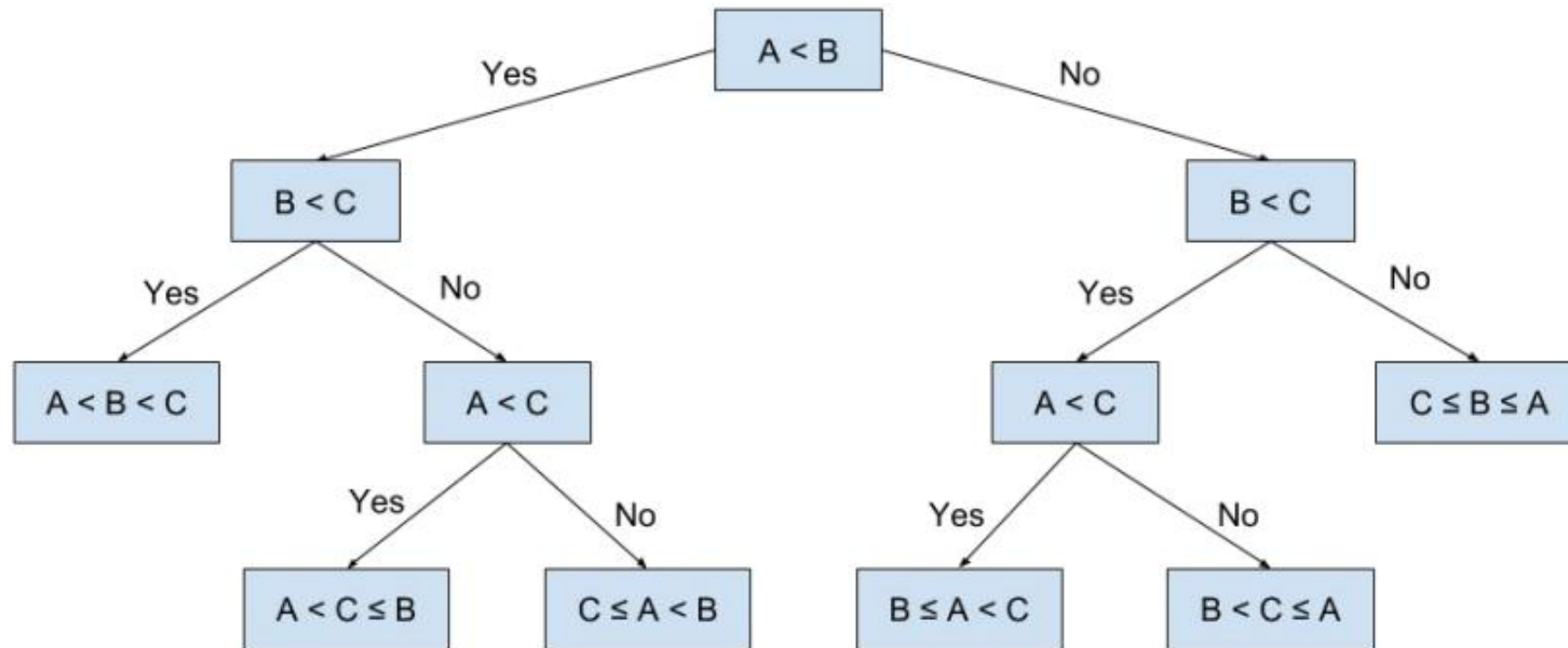
Overfitting



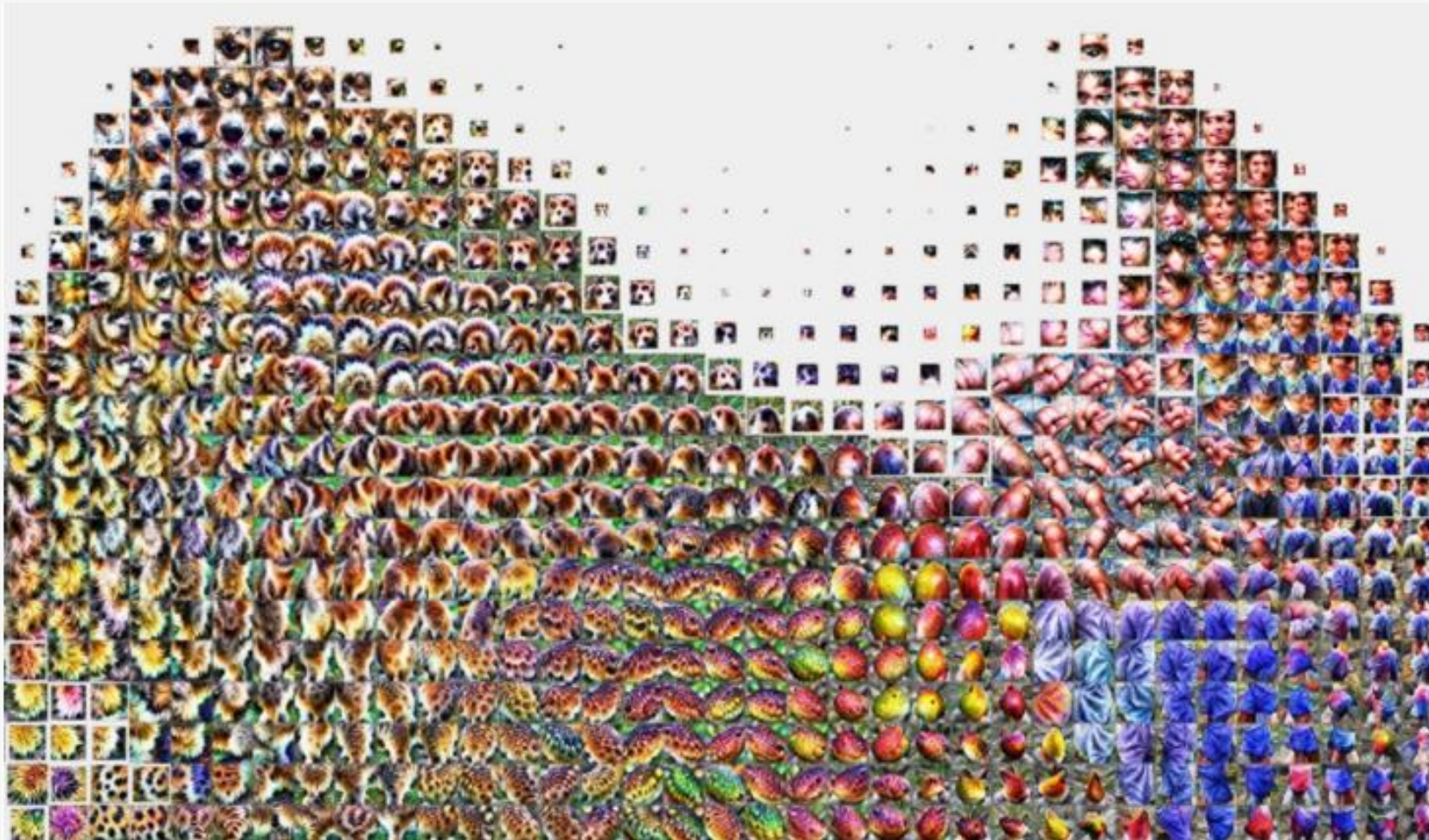
Caja negra



Caja negra



Caja negra



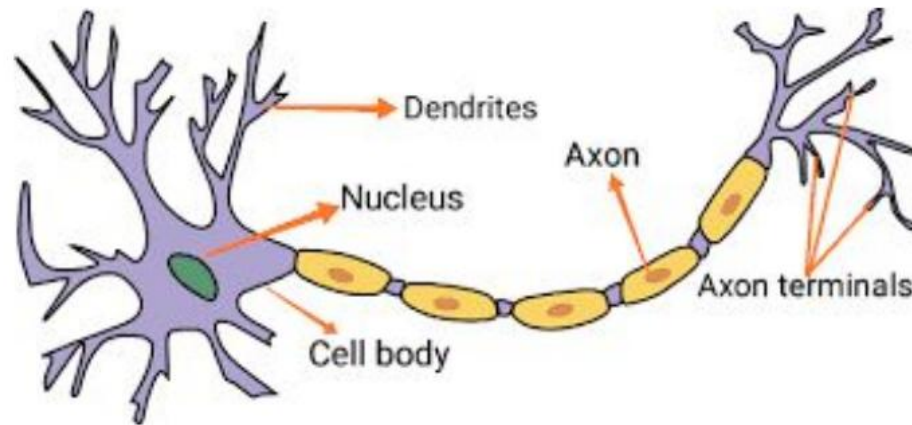
Caja negra



https://adamharley.com/nn_vis/cnn/3d.html

Perceptrón

Redes neuronales biológicas



VS

Redes neuronales artificiales

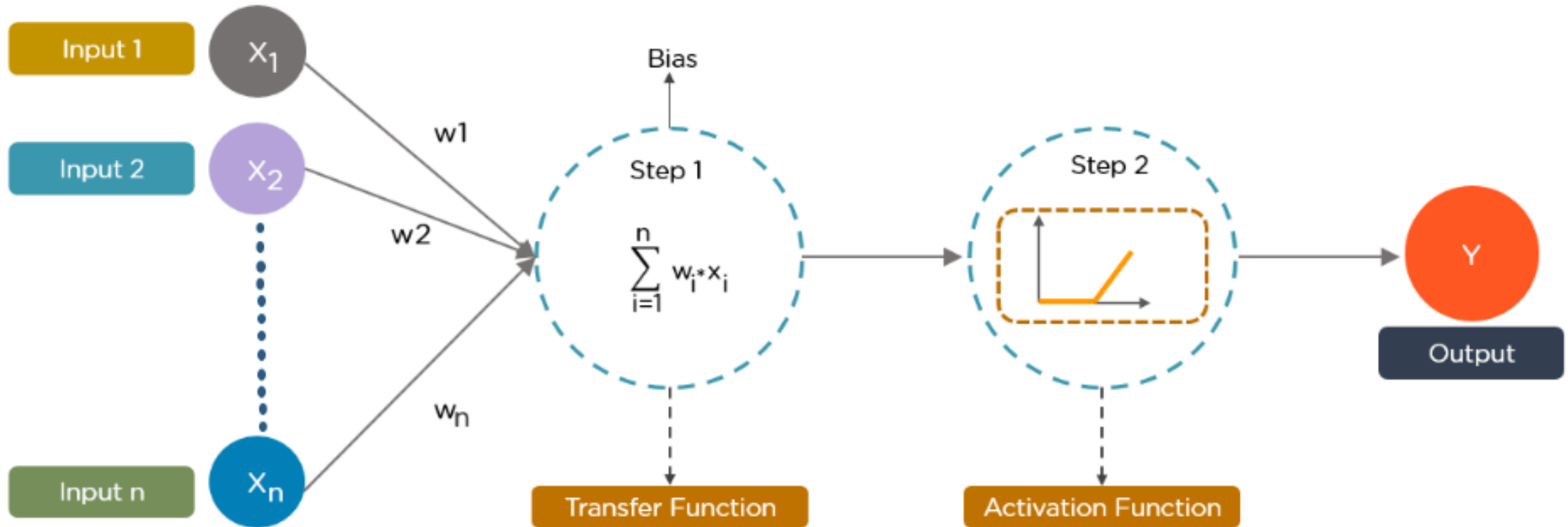


Redes neuronales biológicas:

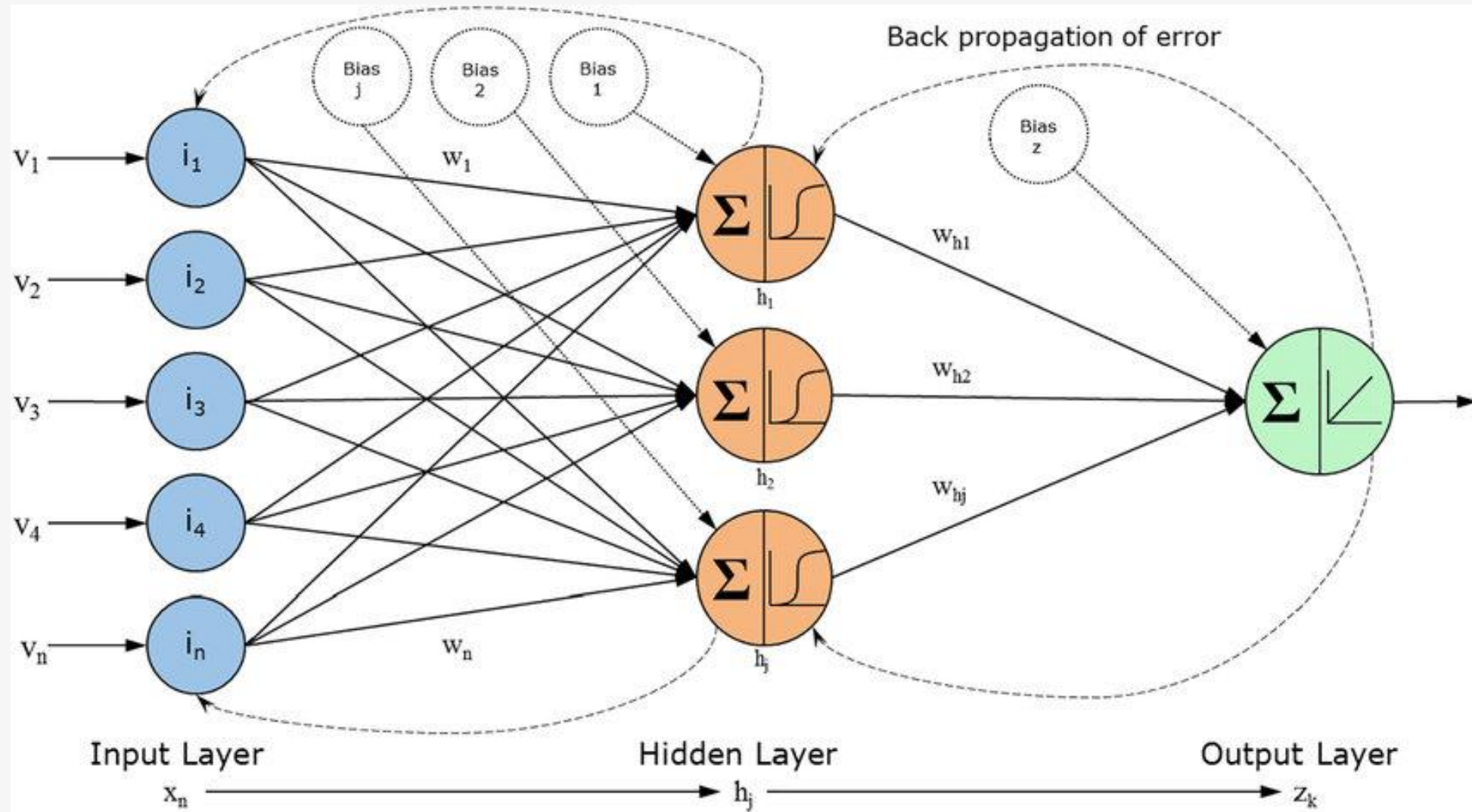
- "Unidad computacional" elemental del cerebro.
- El cerebro tiene aproximadamente 85 mil millones de neuronas.
- Las neuronas interactúan y activan señales eléctricas.
- Aprenden a cuando activar ciertas entradas.

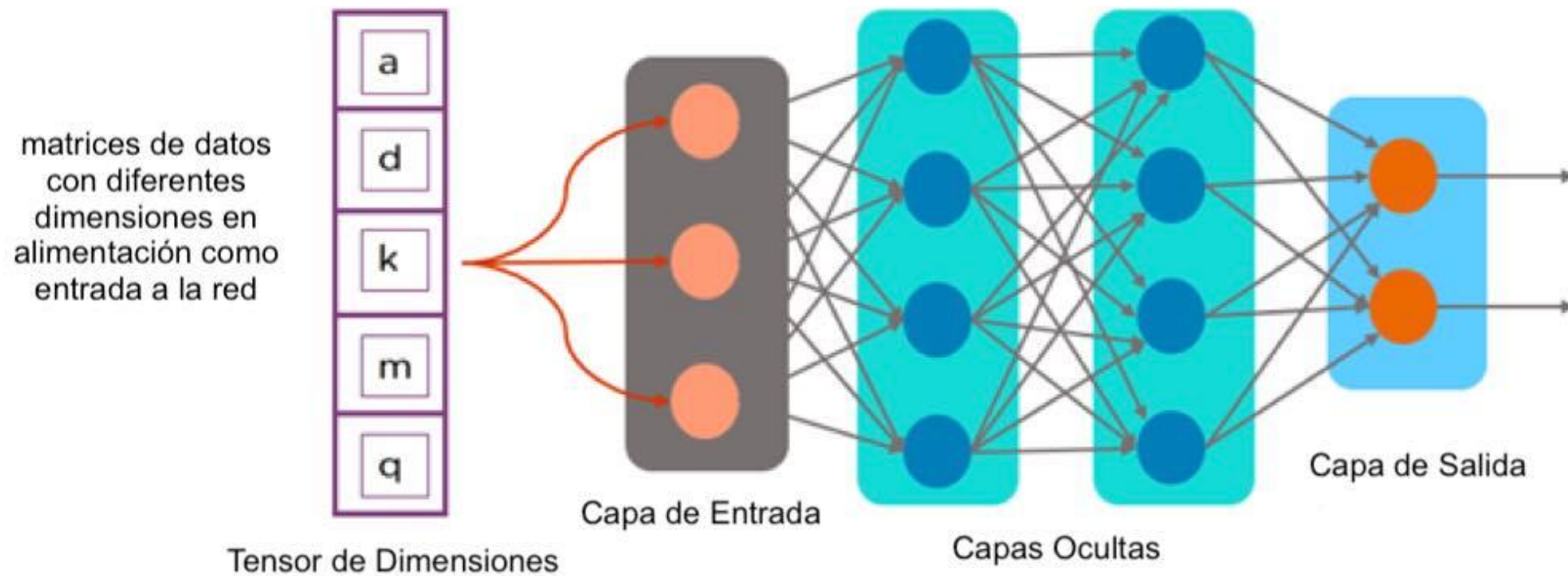
Perceptrón

Modelo matemático simple inventado por Rosenblatt en 1958.

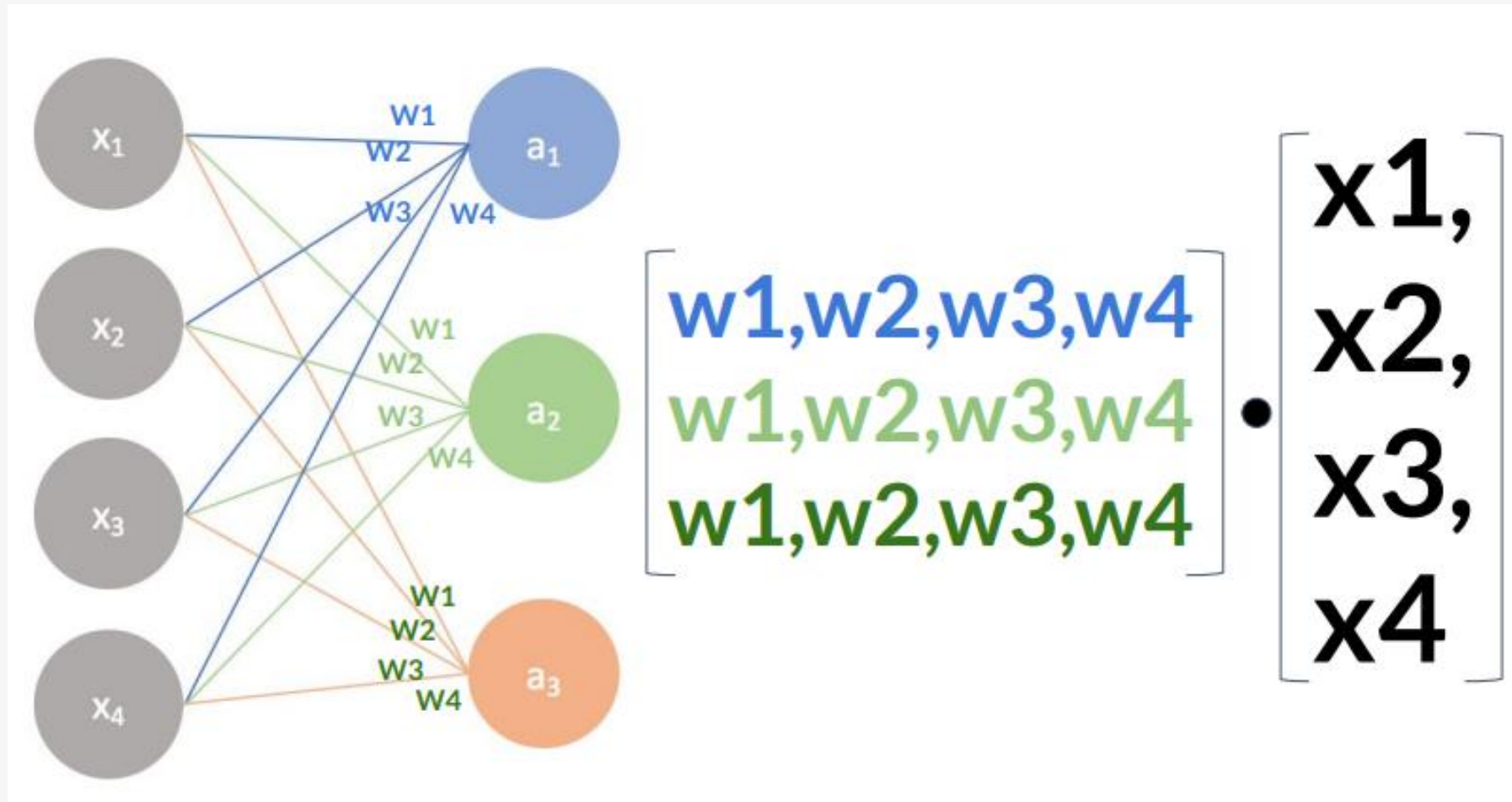


Vamos a Colab





Redes neuronales - Vectores



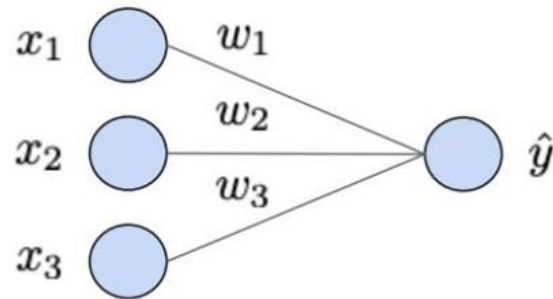
Redes neuronales - Vectores

$$\begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

Conceptos básicos

Caso: 1 capa completamente conectada

1) Dado una entrada x_1, x_2, x_3 , predecir y .



$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b$$

2) Parámetros la red neuronal

$$W = \{[w_1, w_2, w_3], b\}$$

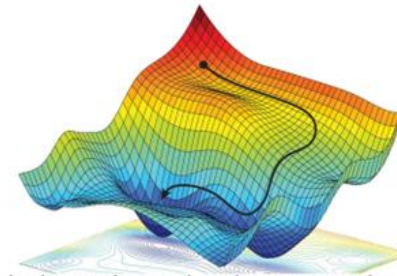
3) Función de costo:

Por cada entrada: $L^i(W) = (\hat{y}^i - y^i)^2$

Por todas las entradas M : $L = \frac{1}{M} \sum_i L^i(W)$

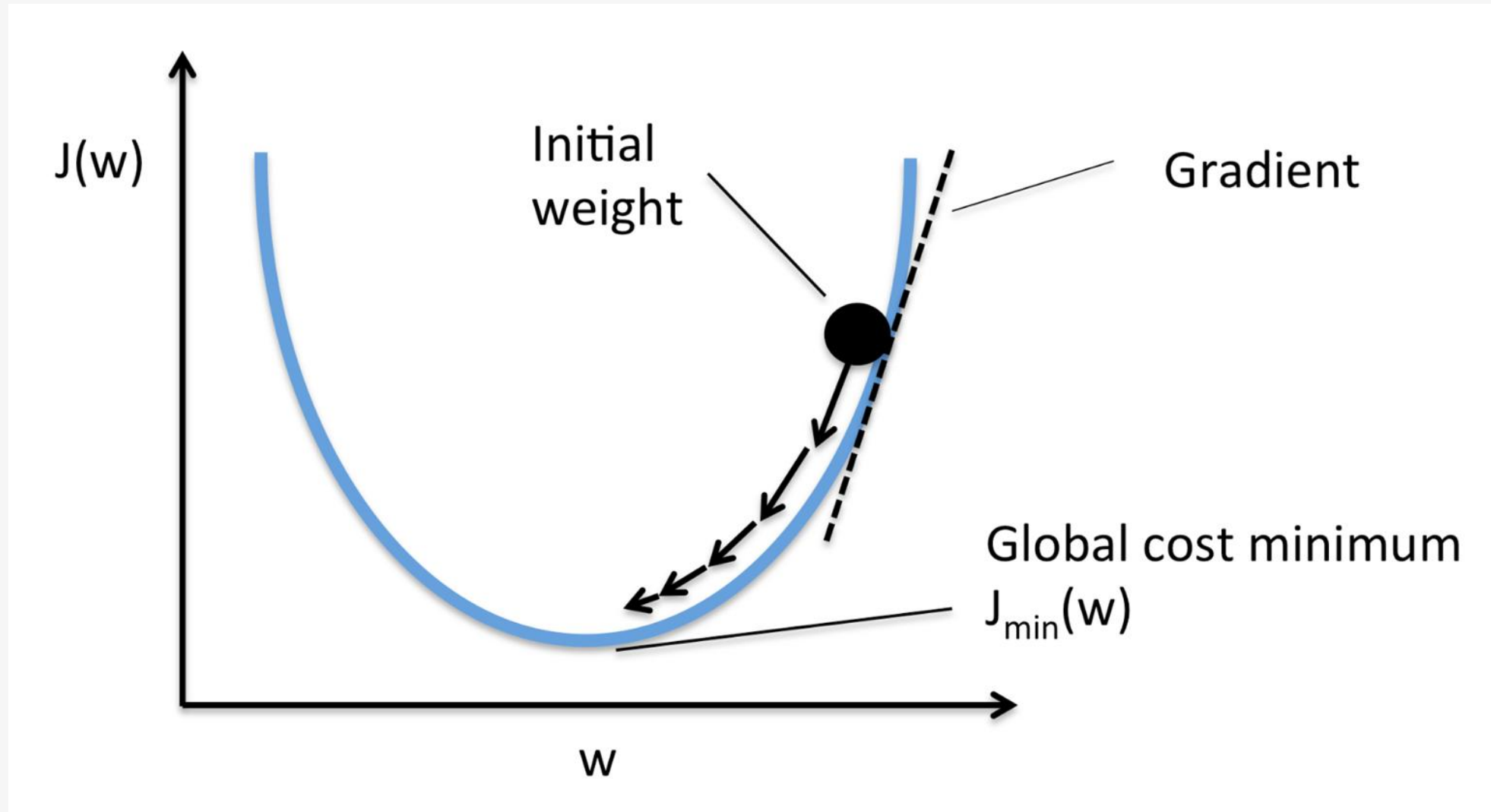
4) Optimización:

¿Por qué? Poder encontrar los parámetros que minimizan mi función de costo!



Leer sobre: regla de la cadena, backpropagation, y gradientes si estás más interesado!

Gradient Descent



Gradient Descent

$$f(x) = x^2$$

1. **Inicialización:** Empezamos con un valor inicial para x , por ejemplo, $x = 3$.
2. **Cálculo del gradiente:** Calculamos el gradiente de $f(x)$ en el punto $x = 3$. La derivada de $f(x)$ es $f'(x) = 2x$, por lo que en $x = 3$, el gradiente es $f'(3) = 2 \cdot 3 = 6$.
3. **Actualización de parámetros:** Usamos el gradiente para actualizar x en la dirección opuesta al gradiente multiplicado por una tasa de aprendizaje. Supongamos que la tasa de aprendizaje es $\alpha = 0.1$. La actualización sería:

$$x_{\text{nuevo}} = x_{\text{viejo}} - \alpha \cdot f'(x_{\text{viejo}}) = 3 - 0.1 \cdot 6 = 2.4$$

Gradient Descent

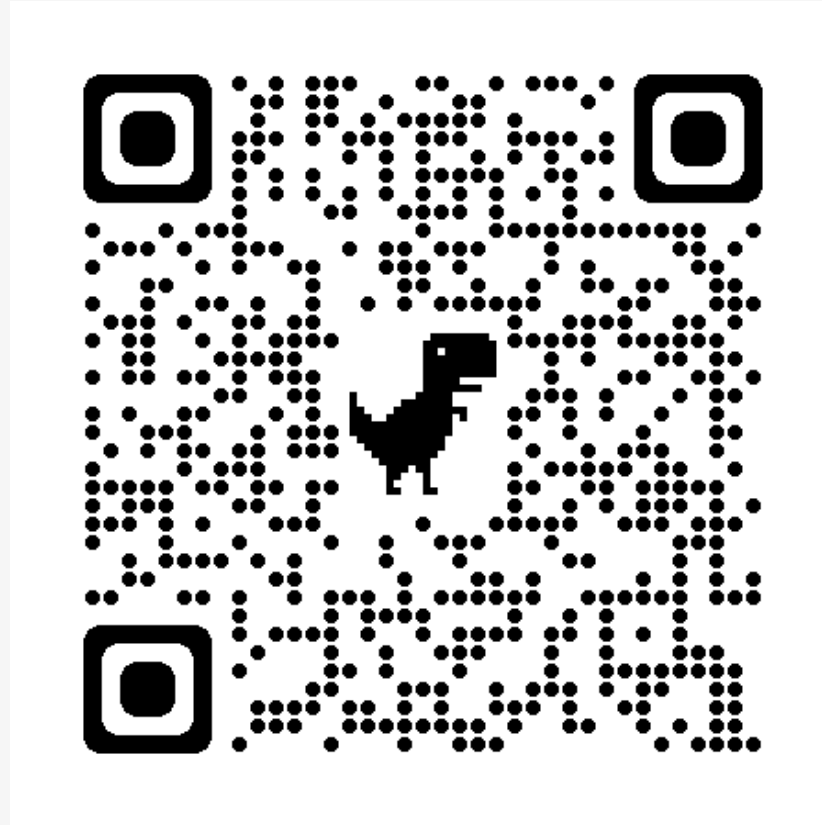
4. **Convergencia:** Repetimos estos pasos hasta que el cambio en x sea muy pequeño o después de un número fijo de iteraciones.

Siguiendo este proceso, en la siguiente iteración tendríamos:

$$x_{\text{nuevo}} = 2.4 - 0.1 \cdot 4.8 = 2.16$$

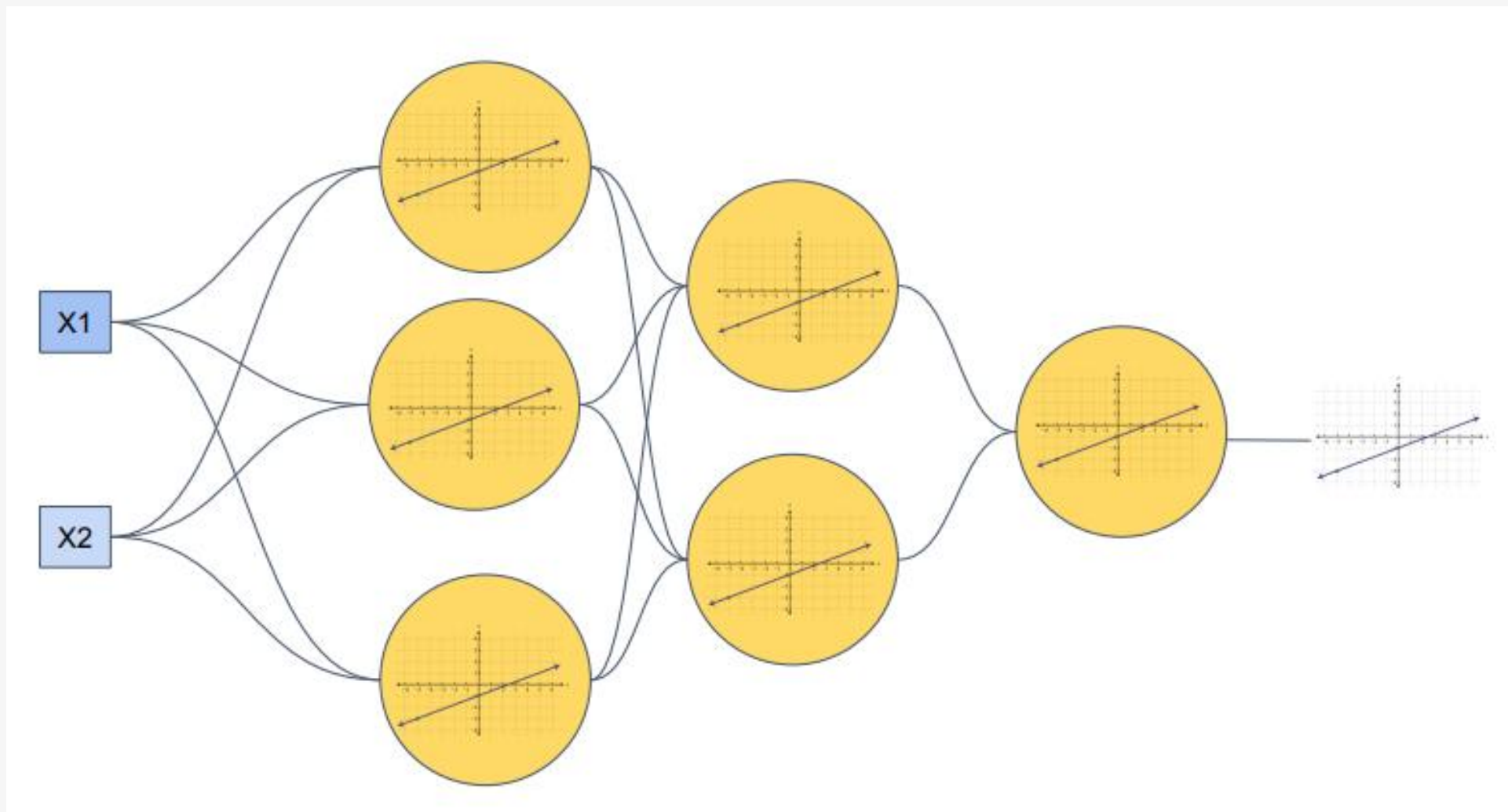
Y así sucesivamente hasta que x converja a un valor cercano a 0, que es el mínimo de la función $f(x) = x^2$.

Tasa de aprendizaje – learning rate

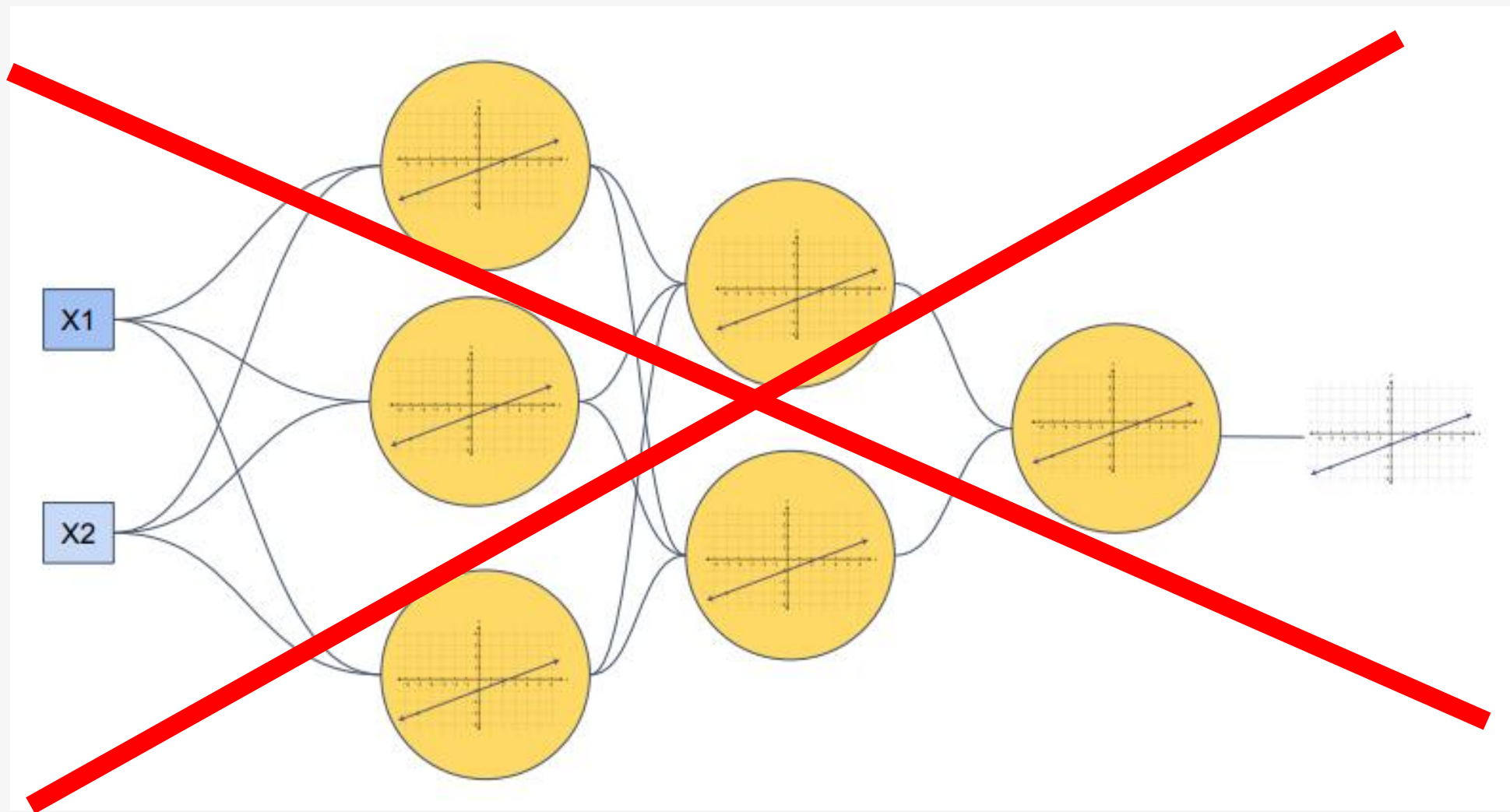


<https://www.benfrederickson.com/numerical-optimization/>

Redes neuronales - Vectores



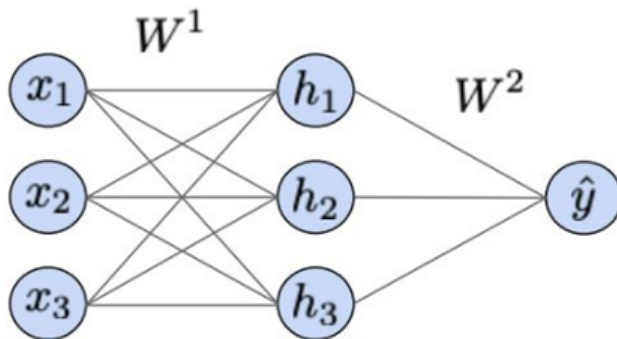
Redes neuronales - Vectores



Conceptos básicos

Caso: 2 capas completamente conectada

1) Dado una entrada x_1, x_2, x_3 , predecir y .



$$\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$$

2) Parámetros la red neuronal

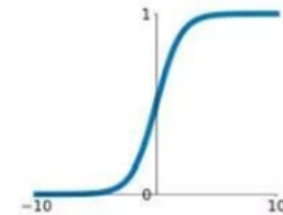
$$W = \{W^1, b^1, W^2, b^2\}$$

$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$
$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \end{bmatrix} \quad b^2 = \begin{bmatrix} b_1^2 \end{bmatrix}$$

Funciones de activación

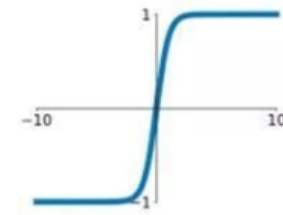
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



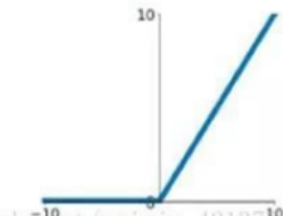
tanh

$$\tanh(x)$$



ReLU

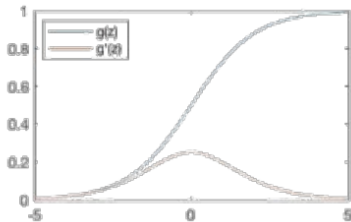
$$\max(0, x)$$



https://blog.csdn.net/weixin_42137100

Funciones de activación

Función Sigmoidea

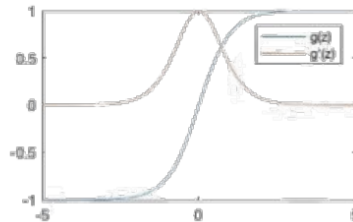


$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

`tf.math.sigmoid(z)`

Tangente Hiperbólica

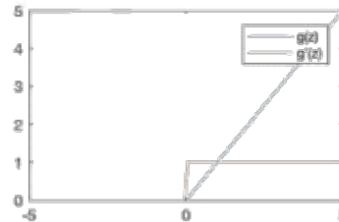


$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

`tf.math.tanh(z)`

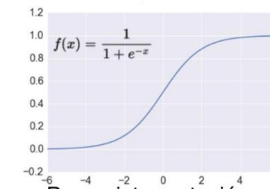
Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

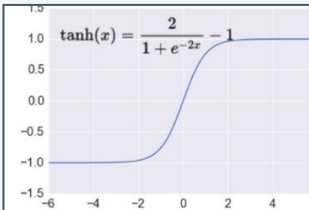
`tf.nn.relu(z)`



Toma un número de valor real y lo "transforma" en un rango entre 0 y 1.

$$\mathbb{R}^n \rightarrow [0, 1]$$

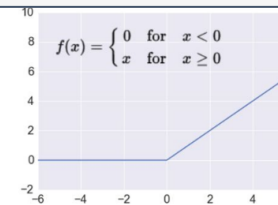
- + Buena interpretación como la tasa de activación de una neurona.
 - 0 = no dispara en absoluto
 - 1 = dispara de forma completa
- Las neuronas sigmoideas saturan y eliminan los gradientes, por lo que las redes apenas aprenderán
 - cuando la activación de la neurona es 0 o 1 (saturado)
 - gradiente en estas regiones casi cero
 - casi ninguna señal fluirá a sus pesos



Toma un número de valor real y lo "transforma" en un rango entre -1 y 1.

$$\mathbb{R}^n \rightarrow [-1, 1]$$

- Como sigmoide, las neuronas tanh se saturan
- A diferencia del sigmoide, la salida está centrada en cero
- Tanh es un sigmoide escalado: $\tanh(x) = 2\text{sigm}(2x) - 1$



Toma un número de valor real y lo establece en cero. $f(x) = \max(0, x)$

$$\mathbb{R}^n \rightarrow \mathbb{R}_+^n$$

<http://adilmoujahid.com/images/activation.png>

La mayoría de las redes profundas utilizan ReLU hoy en día
Entrena mucho más rápido

- acelera la convergencia de SGD
- debido a la forma lineal, no satura

Operaciones menos costosas

- en comparación con sigmoide / tanh (exponenciales, etc.)
- implementado simplemente estableciendo el umbral de una matriz en cero

Más expresiva

Evita el problema de la desaparición del gradiente

Funciones de activación

La función sigmoide se define como:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

La derivada de la función sigmoide con respecto a z es:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

Cuando z es un número grande (positivo o negativo), la salida de la función sigmoide $\sigma(z)$ tiende a 1 o 0, respectivamente. Por ejemplo, si $z = 10$, entonces $\sigma(10) \approx 1$, y si $z = -10$, entonces $\sigma(-10) \approx 0$.

Funciones de activación

Ahora, veamos la derivada $\sigma'(z)$ en estos casos:

- Cuando $z = 10$, $\sigma(10) \approx 1$ y $\sigma'(10) = \sigma(10) \cdot (1 - \sigma(10)) \approx 1 \cdot (1 - 1) = 0$. Aquí, la derivada es muy pequeña debido a que la función sigmoide está saturada en el extremo superior de su rango.
- Cuando $z = -10$, $\sigma(-10) \approx 0$ y $\sigma'(-10) = \sigma(-10) \cdot (1 - \sigma(-10)) \approx 0 \cdot (1 - 0) = 0$. En este caso, la derivada también es muy pequeña debido a que la función sigmoide está saturada en el extremo inferior de su rango.

Por otro lado, cuando z es cercano a 0, la derivada $\sigma'(z)$ es máxima. Por ejemplo, si $z = 0$, entonces $\sigma(0) = 0.5$ y $\sigma'(0) = \sigma(0) \cdot (1 - \sigma(0)) = 0.5 \cdot 0.5 = 0.25$. Aquí, la derivada es mayor porque la función sigmoide no está saturada cerca de 0.

Funciones de activación

Ahora, veamos la derivada $\sigma'(z)$ en estos casos:

- Cuando $z = 10$, $\sigma(10) \approx 1$ y $\sigma'(10) = \sigma(10) \cdot (1 - \sigma(10)) \approx 1 \cdot (1 - 1) = 0$. Aquí, la derivada es muy pequeña debido a que la función sigmoide está saturada en el extremo superior de su rango.
- Cuando $z = -10$, $\sigma(-10) \approx 0$ y $\sigma'(-10) = \sigma(-10) \cdot (1 - \sigma(-10)) \approx 0 \cdot (1 - 0) = 0$. En este caso, la derivada también es muy pequeña debido a que la función sigmoide está saturada en el extremo inferior de su rango.

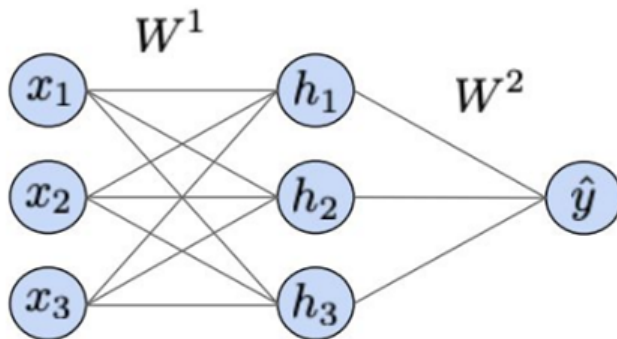
Por otro lado, cuando z es cercano a 0, la derivada $\sigma'(z)$ es máxima. Por ejemplo, si $z = 0$, entonces $\sigma(0) = 0.5$ y $\sigma'(0) = \sigma(0) \cdot (1 - \sigma(0)) = 0.5 \cdot 0.5 = 0.25$. Aquí, la derivada es mayor porque la función sigmoide no está saturada cerca de 0.

$$x_{\text{nuevo}} = x_{\text{viejo}} - \alpha \cdot f'(x_{\text{viejo}})$$

Conceptos básicos

Caso: 2 capas completamente conectada

1) Dado una entrada x_1, x_2, x_3 , predecir y .



$$\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$$

2) Parámetros la red neuronal

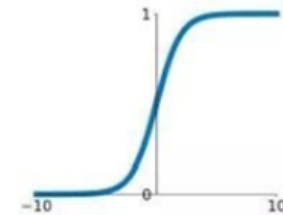
$$W = \{W^1, b^1, W^2, b^2\}$$

$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$
$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \end{bmatrix} \quad b^2 = \begin{bmatrix} b_1^2 \end{bmatrix}$$

Funciones de activación

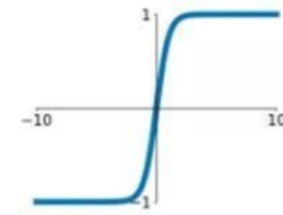
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh

$$\tanh(x)$$



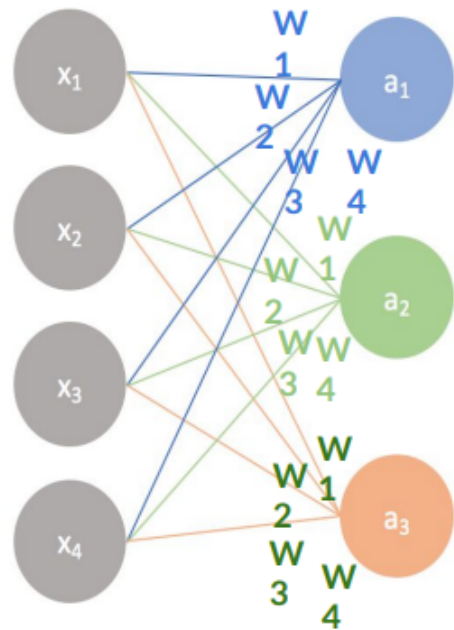
ReLU

$$\max(0, x)$$



https://blog.csdn.net/weixin_42137100

Funciones de activación - softmax



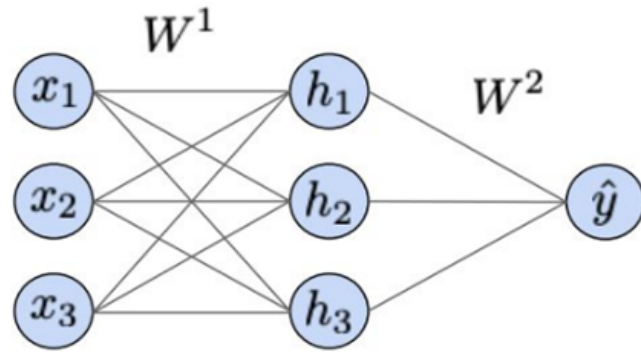
$$\mathbf{z} = \begin{bmatrix} 1.6 \\ 0.55 \\ 0.98 \end{bmatrix} = \begin{bmatrix} 0.51 \\ 0.18 \\ 0.31 \end{bmatrix}$$

$$y_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

Conceptos básicos

Caso: 2 capas completamente conectada

1) Dado una entrada x_1, x_2, x_3 , predecir y .



$$\hat{y} = W^2(\sigma(W^1x + b^1)) + b^2$$

2) Parámetros la red neuronal

$$W = \{W^1, b^1, W^2, b^2\}$$

$$W^1 = \begin{bmatrix} w_{11}^1 & w_{12}^1 & w_{13}^1 \\ w_{21}^1 & w_{22}^1 & w_{23}^1 \\ w_{31}^1 & w_{32}^1 & w_{33}^1 \end{bmatrix} \quad b^1 = \begin{bmatrix} b_1^1 \\ b_2^1 \\ b_3^1 \end{bmatrix}$$
$$W^2 = \begin{bmatrix} w_{11}^2 & w_{12}^2 & w_{13}^2 \end{bmatrix} \quad b^2 = \begin{bmatrix} b_1^2 \end{bmatrix}$$

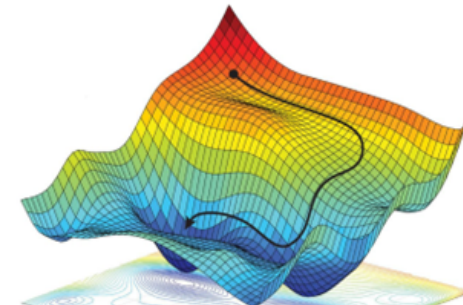
3) Función de costo:

Por cada entrada: $L^i(W) = (\hat{y}^i - y^i)^2$

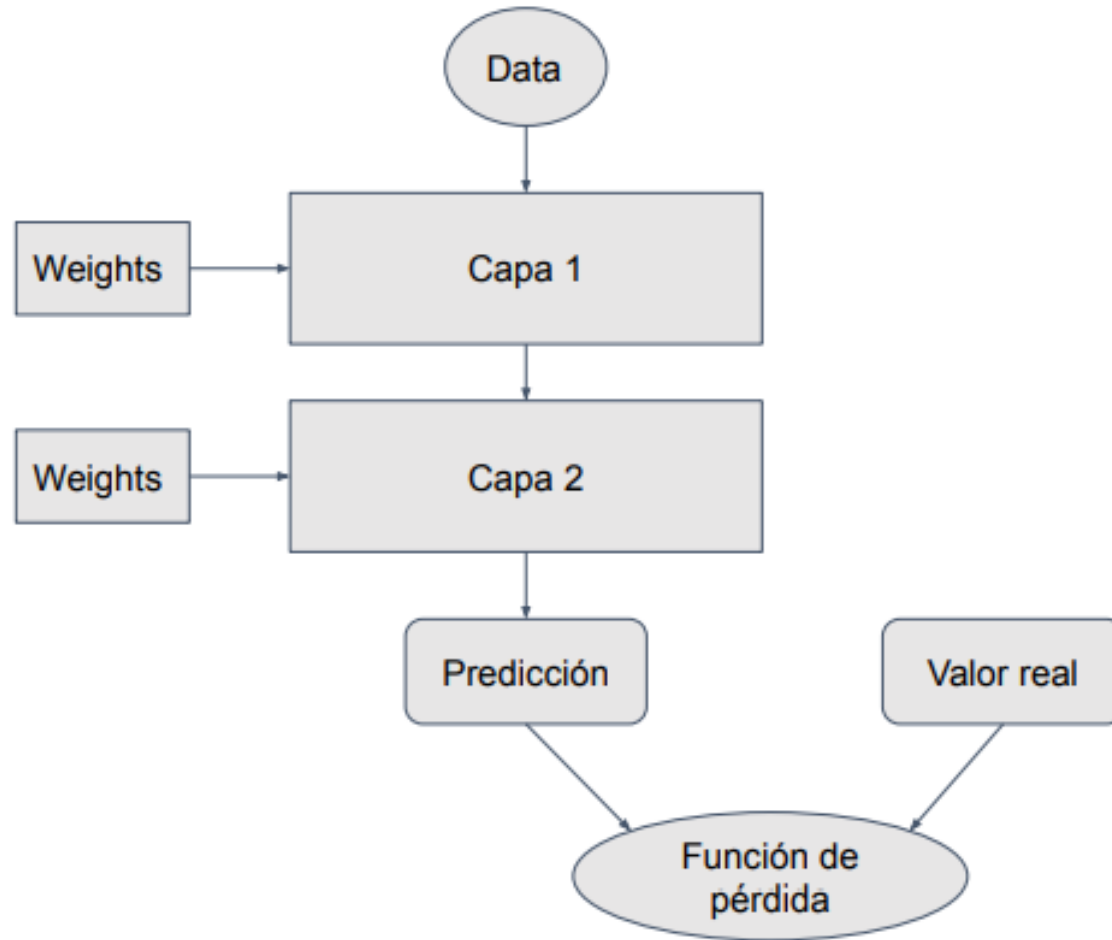
Por todas las entradas M : $L = \frac{1}{M} \sum_i L^i(W)$

4) Optimización:

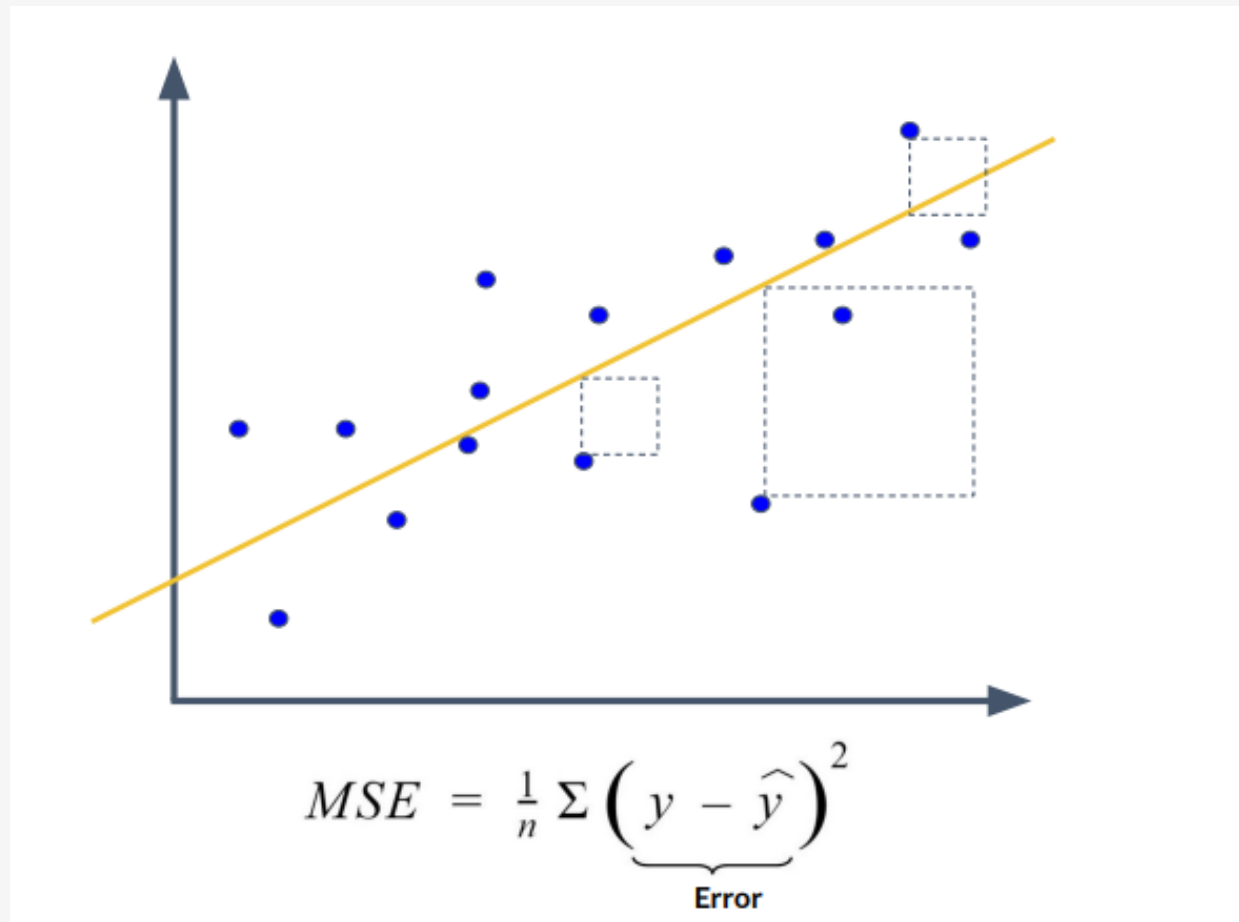
¿Por qué? Poder encontrar los parámetros que minimizan mi función de costo!



Conceptos básicos



MSE – Mean squared error

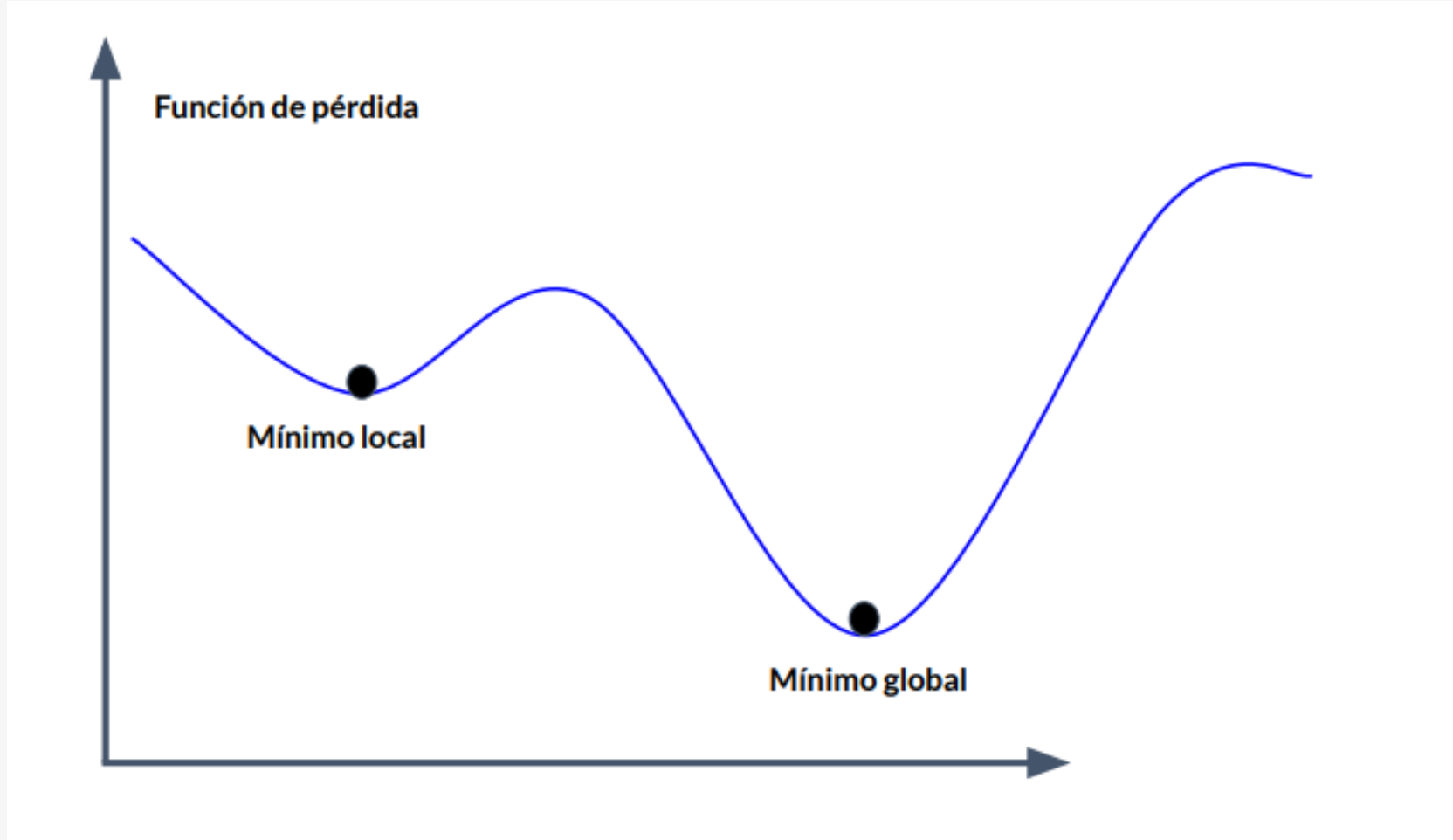


Cross Entropy

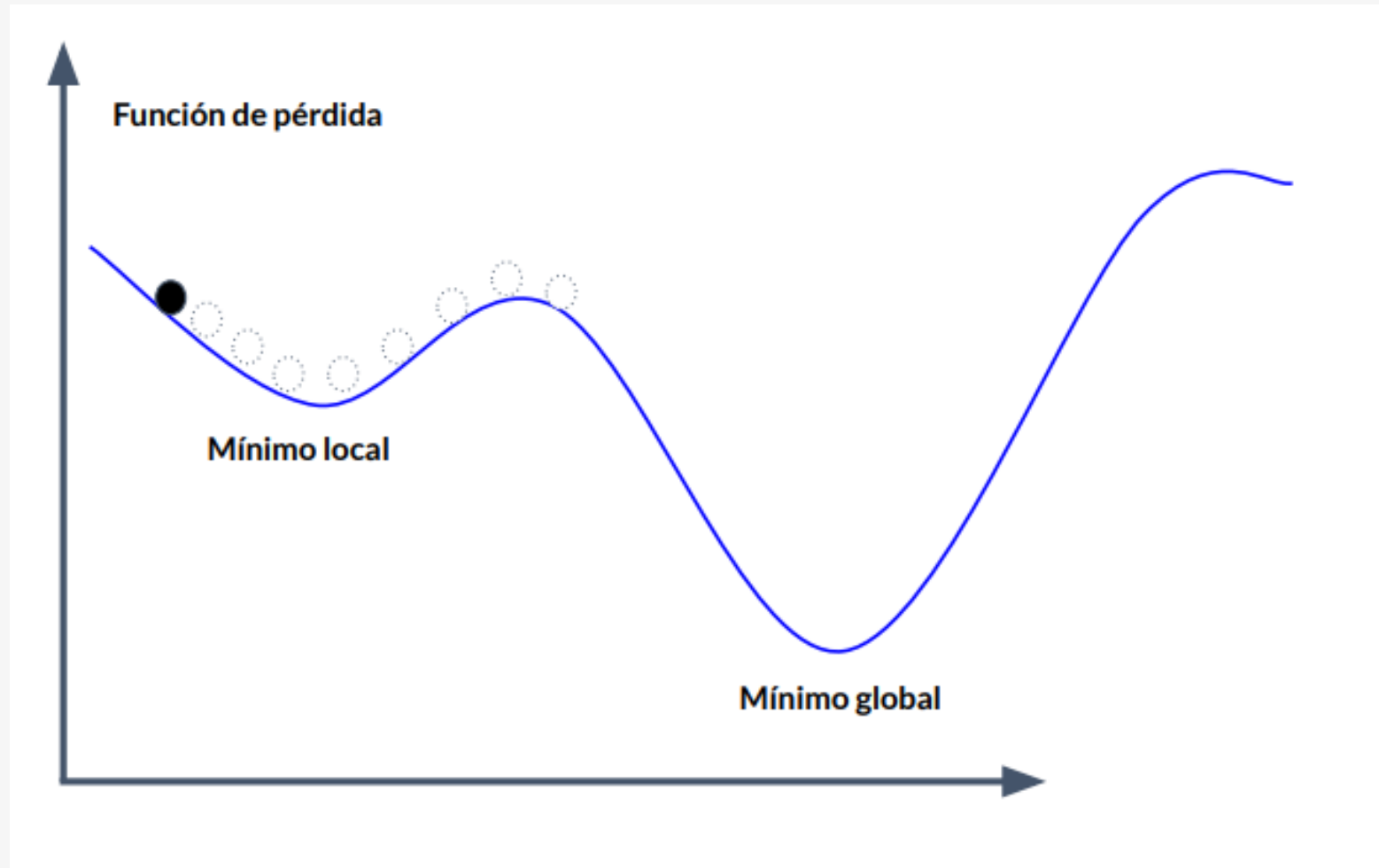
	Verdad	Predicción	
	$\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0.5 \\ 0.3 \\ 0.2 \end{pmatrix}$	$\begin{matrix} = \\ = \end{matrix} \begin{matrix} \text{Log}(0.5) = -0,301029996 \\ p(x_0) = 1 \end{matrix} \begin{matrix} = \\ = \end{matrix} 0,301029996$
			
			

$$-\sum_x p(x) \log q(x).$$

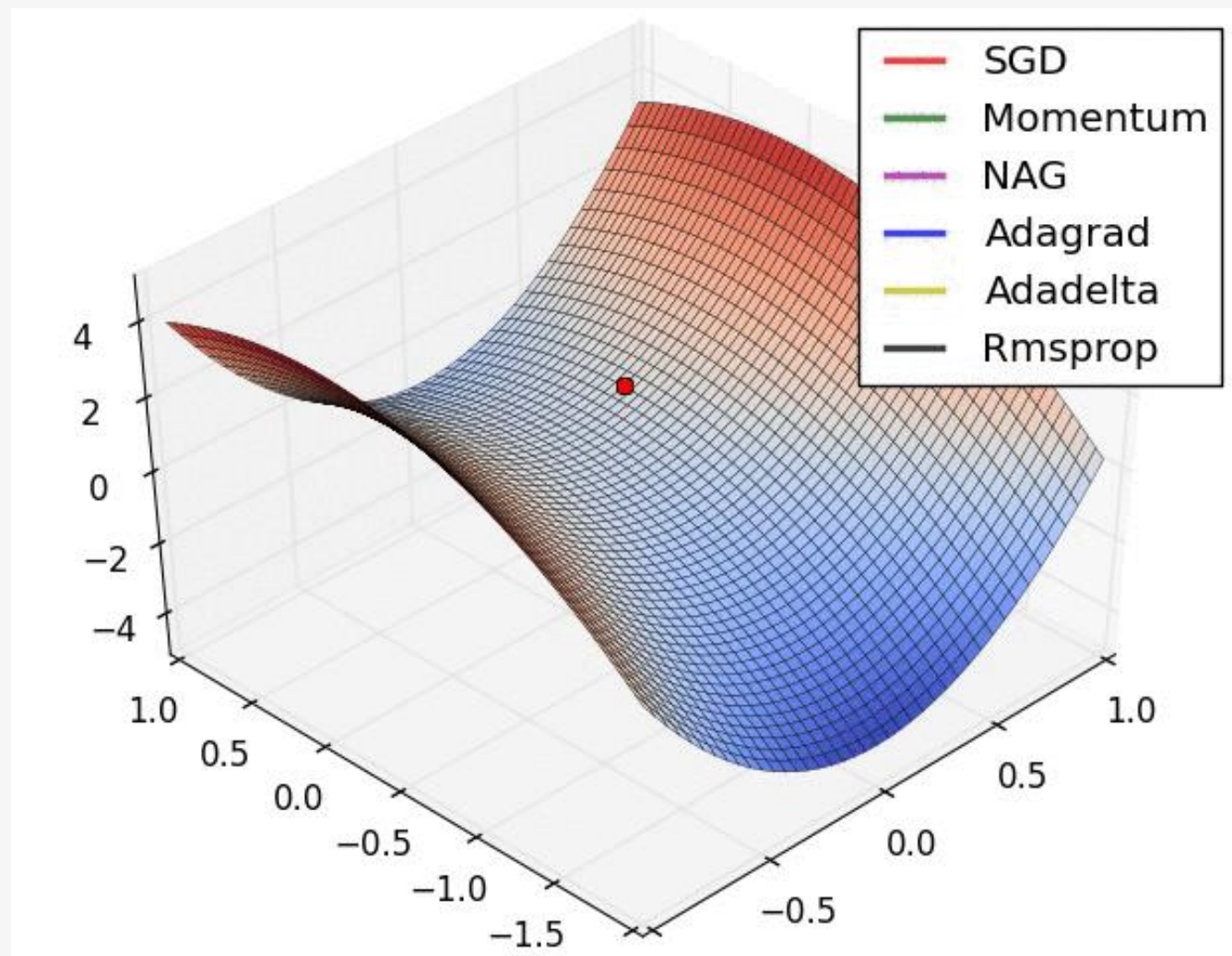
Gradient Descent



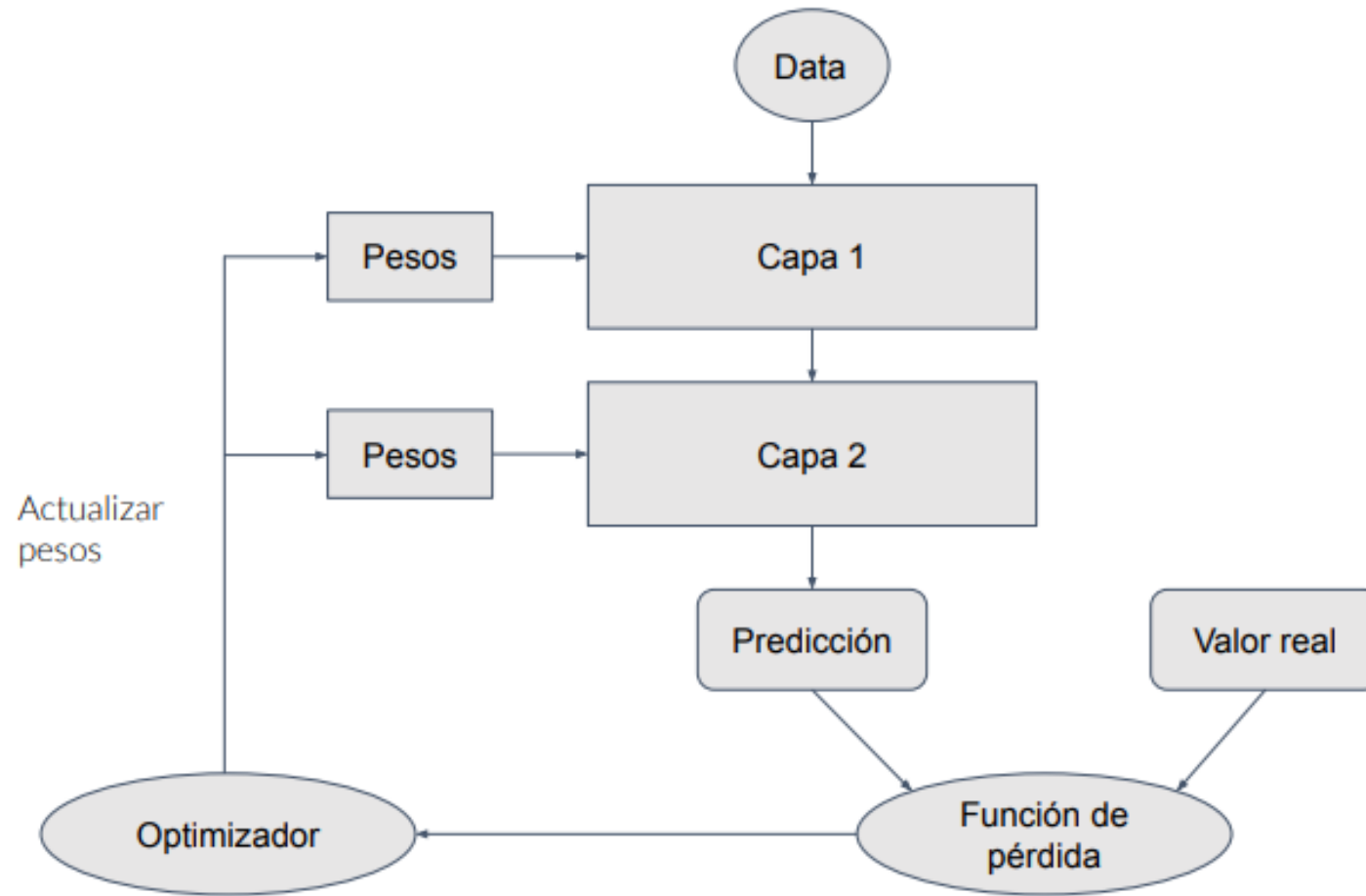
Momentum



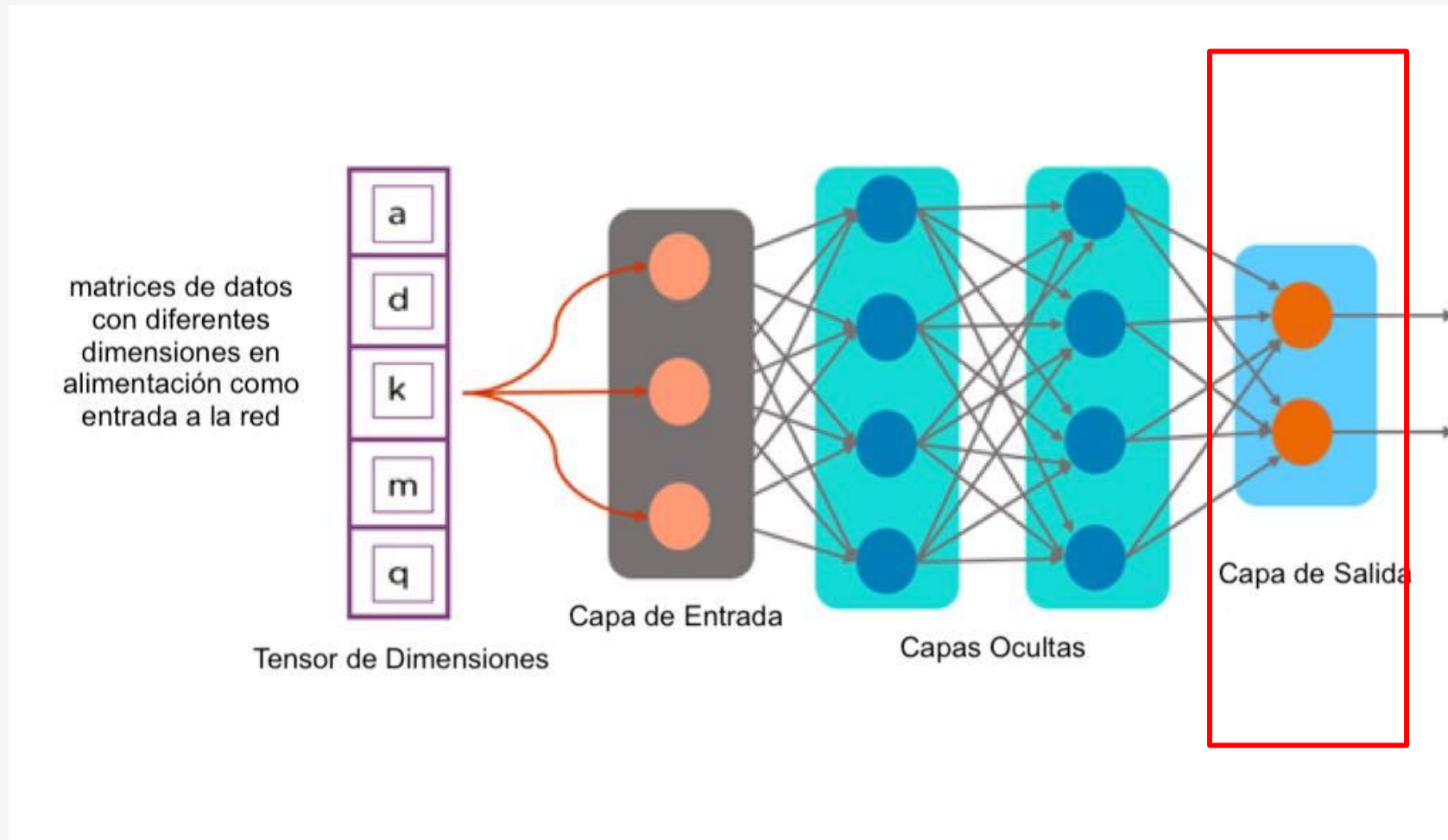
Optimizadores



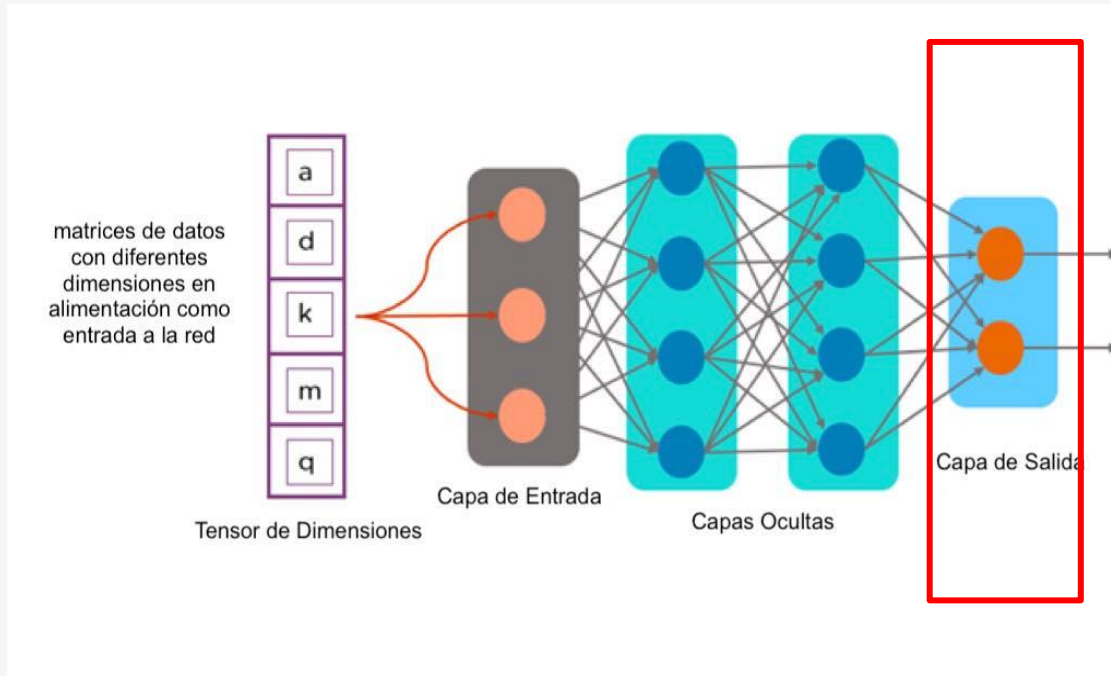
Backpropagation



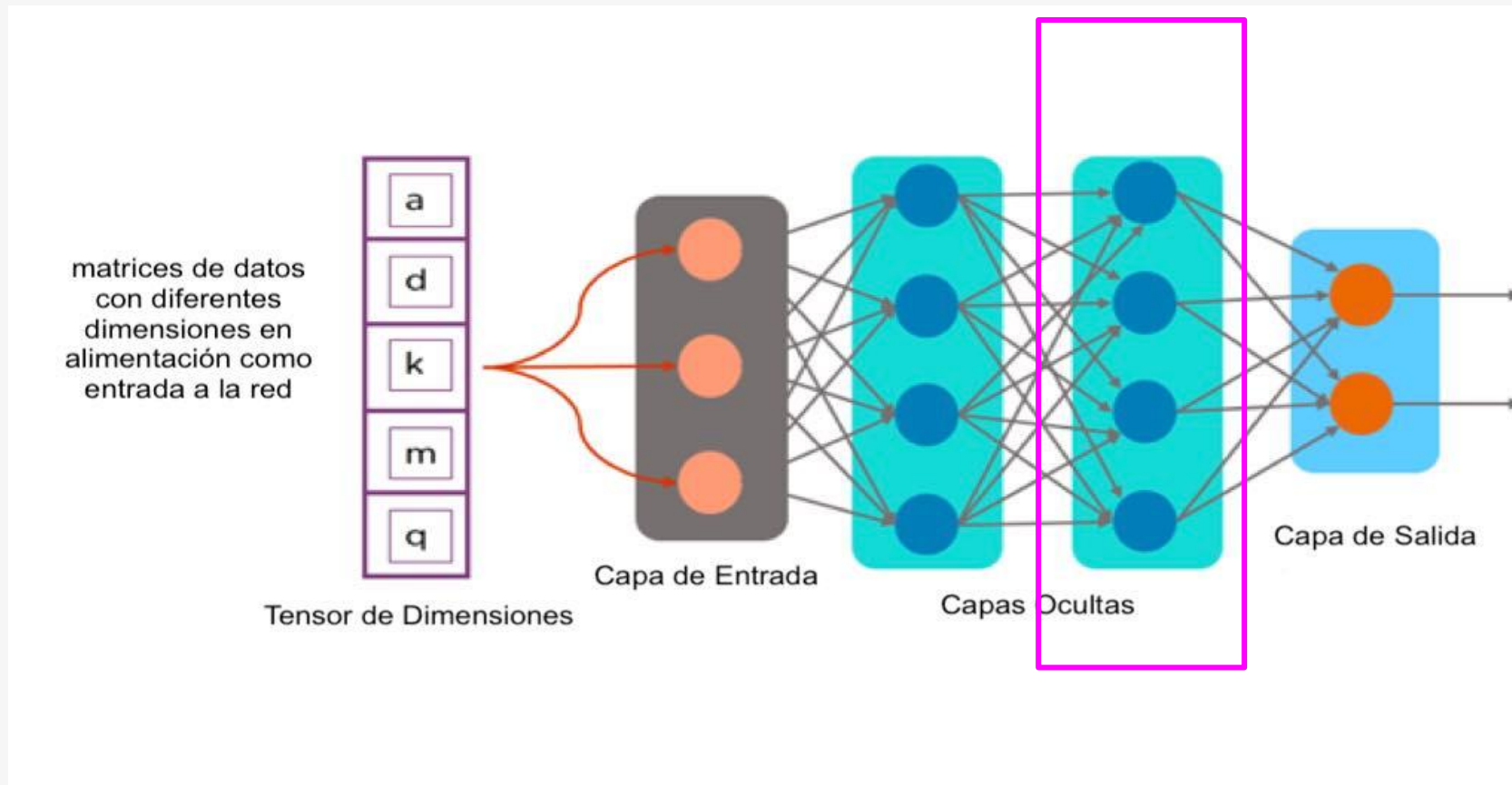
Backpropagation



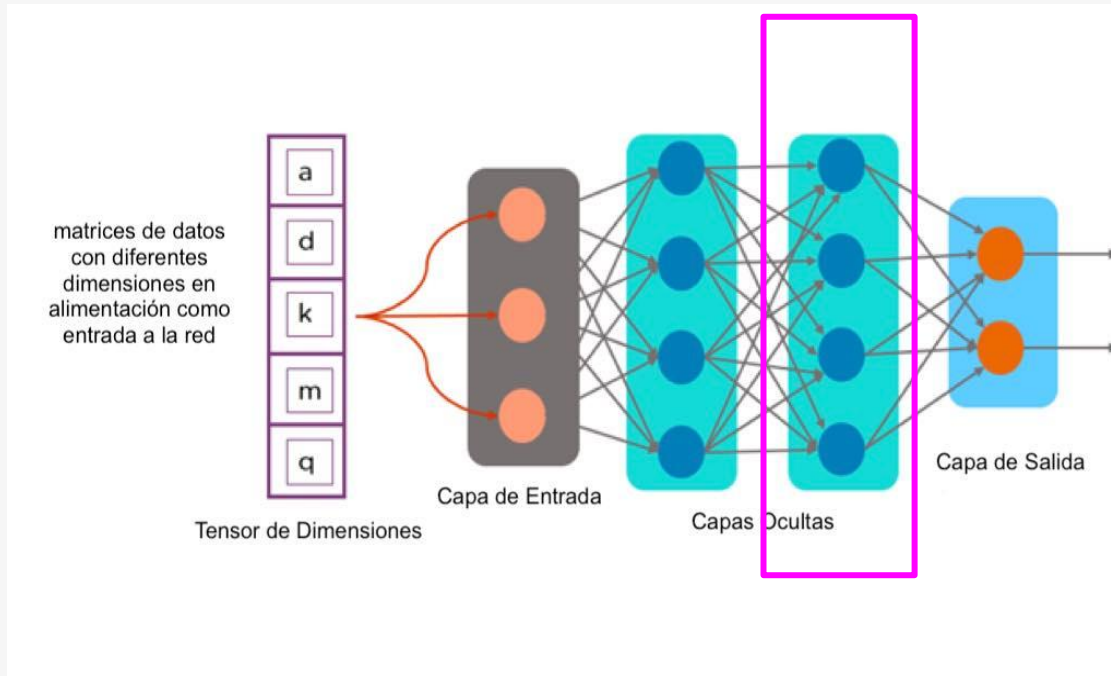
Backpropagation



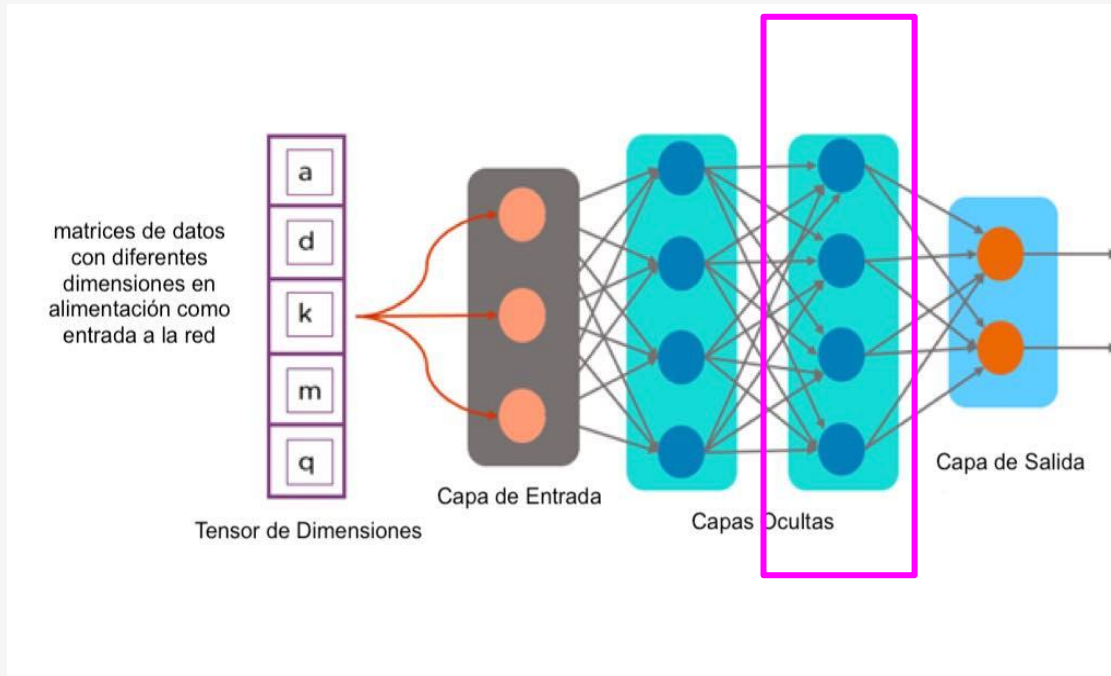
Backpropagation



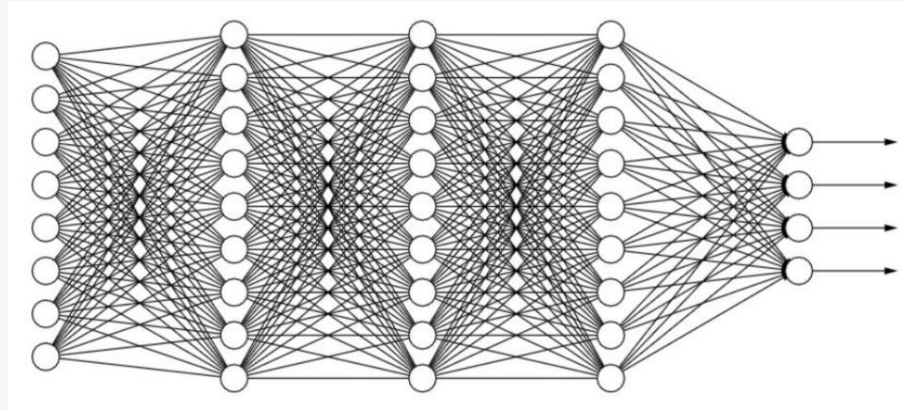
Backpropagation



Backpropagation



Backpropagation



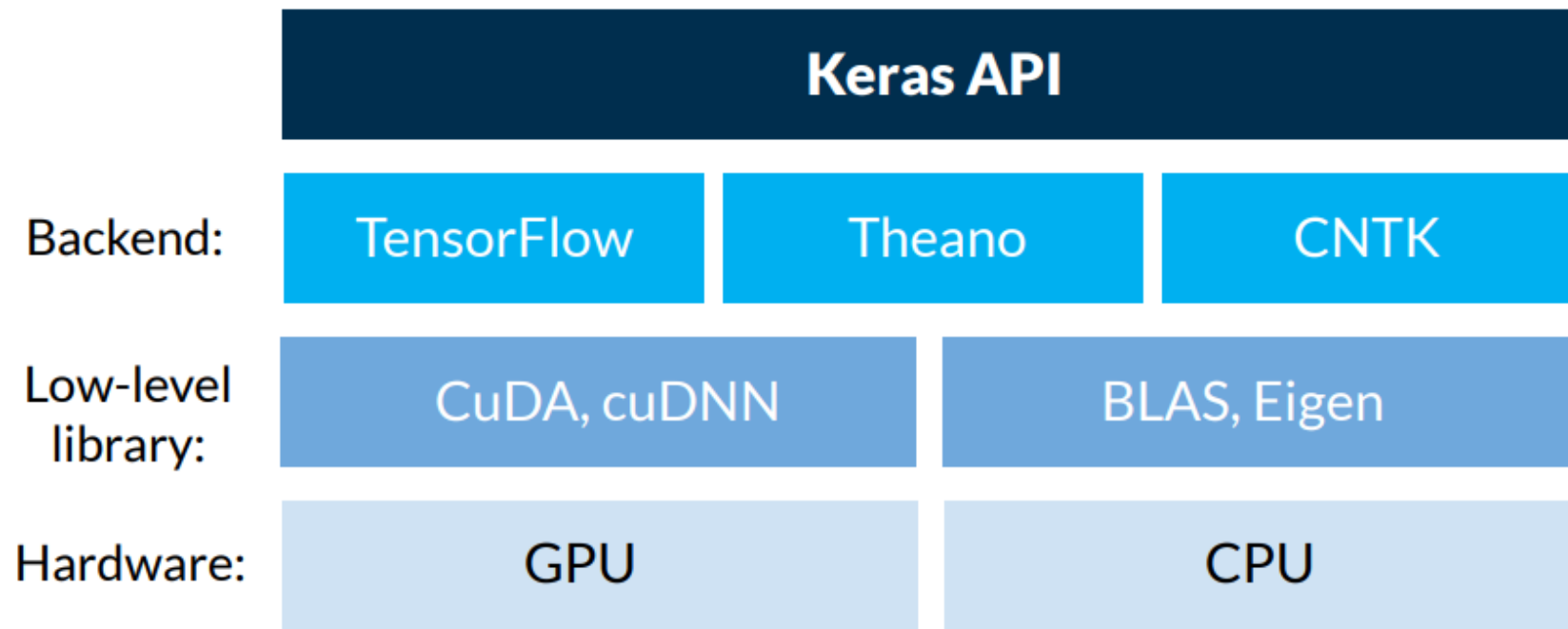
$$\delta[l] \equiv \text{Costo(Activación}(z)) \equiv \frac{\partial C}{\partial a[l]} * \frac{\partial a[l]}{\partial z[l]}$$

$$\delta[l-1] \equiv W[l] \delta[l] * \frac{\partial a[l-1]}{\partial z[l-1]}$$

Error

$$\delta[l-1] * a[l-2]$$

Introducción a Keras



Keras

- François Chollet es el autor de Keras.
 - Actualmente trabaja para Google como ingeniero e investigador de deep learning.
- Keras significa cuerno en griego
- Documentación: <http://keras.io/>
- Ejemplo: <https://github.com/fchollet/keras/tree/master/examples>

Keras

```
$ pip install tensorflow
```



```
$ pip install keras
```

Usar el código:

```
import keras  
print (keras.__version__)
```

Instalación de tensorflow en raspberry pi

<https://github.com/PINTO0309/Tensorflow-bin>

en pc:

<https://docs.anaconda.com/anaconda/user-guide/tasks/tensorflow/>

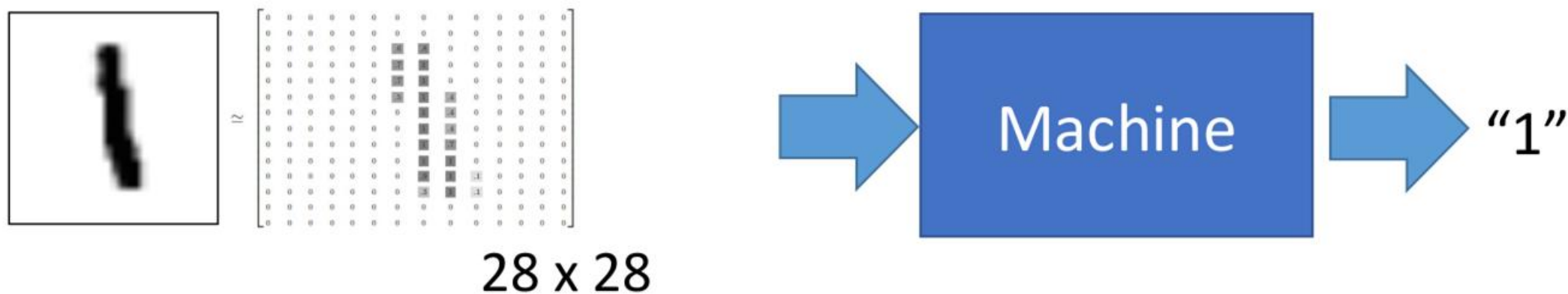
<https://anaconda.org/conda-forge/tensorflow>

<https://stackoverflow.com/questions/59232286/error-cannot-uninstall-wrapt-when-installing-tensorflow-gpu-1-14>

Datasets famosos

- 1936 → IRIS flower dataset: 50 muestras (4 características) y 3 categorías
- 1986 → TIMIT speech transcription: 6,300 muestras (texto - transcripción)
- 1987 → STATLOG credits: 690 muestras y 2 categorías
- 1988 → SOYBEAN plants: 307 muestras (35 características) y 19 cate^gegs.
- 1998 → MNIST digits: 70,000 muestras (28x28 B/N) y 10 categorías
- 2003 → CALTECH objects: 9,146 muestras (300x200) y 101 categorías
- 2003 → FERET faces: 11,338 muestras y 1,199 categorías
- 2009 → CIFAR 10 objects: 60,000 muestras (32x32) y 10 categorías
- 2014 → IMAGENET objects: 14,197,122 muestras y 20,000 categorías
- 2015 → COCO objects: 2,500,000 muestras y 91 categorías
- 2015 → VIDRILO* rgbd: 22,454 muestras y 25 categorías (15 objetos y 10 habitaciones)

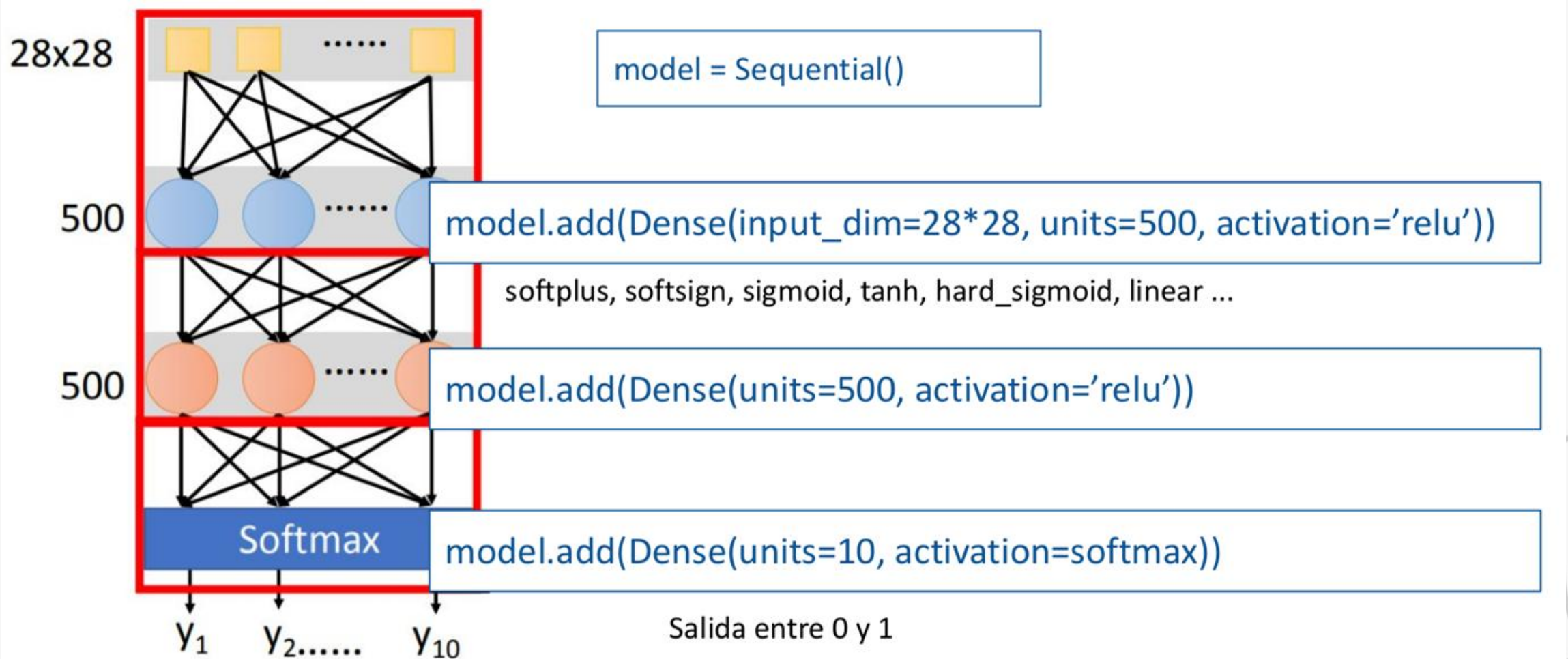
Reconocimiento de dígitos



Datos MNIST: <http://yann.lecun.com/exdb/mnist/>

Keras proporciona la función de carga de conjuntos de datos: <http://keras.io/datasets/>

Construyendo una red en Keras



Configuraciones en Keras

```
model.compile(loss='categorical_crossentropy',  
              optimizer='adam', metrics=['accuracy'])
```

SGD, RMSprop, Adagrad, Adam

Escogiendo la mejor función en Keras

```
model.fit(x_train, y_train, batch_size=100, nb_epoch=20)
```

numpy array

28 x 28
=784

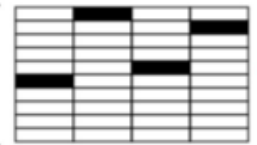


.....

Número de ejemplos de
entrenamiento

numpy array

10



.....

Número de ejemplos de
entrenamiento

Vamos a Colab

Redes Neuronales Convolucionales

Msc. Lewis De La Cruz



cayetano.edu.pe



¿Y si empezamos con un chismecito?





xAI, la startup de Elon Musk de IA, recauda US\$ 6.000 millones

La firma de inteligencia artificial xAI, de Elon Musk, recauda US\$ 6.000 millones en financiación.

Por [Patricia Kaplan](#), CNN Publicado a las 17:50 ET (21:50 GMT) 27 mayo, 2024



Elon Musk ✓ ✖ @elonmusk · May 27



Join xAI if you believe in our mission of understanding the universe, which requires maximally rigorous pursuit of the truth, without regard to popularity or political correctness



Igor Babuschkin ✓ ✖ @ibab · May 27

Apply at x.ai/careers if you want to be part of our journey to build AGI and understand the Universe 🧠🔗

💬 8K

↻ 22K

♥ 82K

📊 45M





Elon Musk ✓ [X] @elonmusk · May 27



Join xAI if you believe in our mission of understanding the universe, which requires maximally rigorous pursuit of the truth, without regard to popularity or political correctness



Igor Babuschkin ✓ [X] @ibab · May 27

Apply at x.ai/careers if you want to be part of our journey to build AGI and understand the Universe 🧩👉



8K



22K



82K



45M



Yann LeCun ✓ [∞] @ylecun · May 27



Join xAI if you can stand a boss who:

- claims that what you are working on will be solved next year (no pressure).
- claims that what you are working on will kill everyone and must be stopped or paused (yay, vacation for 6 months!).
- claims to want a "maximally rigorous pursuit

[Show more](#)



1K



1.2K



12K



1.1M





Elon Musk ✓ [X] @elonmusk · May 27



Join xAI if you believe in our mission of understanding the universe, which requires maximally rigorous pursuit of the truth, without regard to popularity or political correctness



Igor Babuschkin ✓ [X] @ibab · May 27

Apply at x.ai/careers if you want to be part of our journey to build AGI and understand the Universe 🧩👉



8K



22K



82K



45M



Yann LeCun ✓ [∞] @ylecun · May 27



Join xAI if you can stand a boss who:

- claims that what you are working on will be solved next year (no pressure).
- claims that what you are working on will kill everyone and must be stopped or paused (yay, vacation for 6 months!).
- claims to want a "maximally rigorous pursuit

[Show more](#)



1K



1.2K



12K



1.1M





Dave Hayes ✓ @how_many_roads_ · May 27

Not a good look... Maybe explain what you have against Elon?

76

13

1K

425K



Yann LeCun ✓ @ylecun · May 27

I like his cars, his rockets, his solar panels, and his satellite network.
I very much dislike his vengeful politics, his conspiracy theories, and his hype.

423

434

8.6K

512K



Julien Bouteloup ✓ @bneiluj · May 27

Yann, with all the respect I have for your work, and as one of the few who convinced me to learn about machine learning more than 15 years ago, why don't you start your own AI company and try to do good? Staying at Meta and telling others they are wrong is a bit odd, no? 🙏

30

33

1.5K

365K



Yann LeCun ✓ @ylecun · May 27

I'm a scientist, not a business or product person.

164

139

4.8K

480K



Elon Musk ✓ @elonmusk · May 27

What "science" have you done in the past 5 years?

432

546

3.7K

1M





Elon Musk ✓ ✖ @elonmusk · May 27

What "science" have you done in the past 5 years?

432

546

3.7K

1.1M



Yann LeCun ✓ @ylecun · May 27

Over 80 technical papers published since January 2022.
What about you?



scholar.google.com

Yann LeCun

Chief AI Scientist at Facebook & Silver Professor at the
Courant Institute, New York University - Cited by ...

479

978

13K

738K



Elon Musk ✓ ✖ @elonmusk · May 27

That's nothing, you're going soft. Try harder!

597

439

3.7K

514K



Yann LeCun ✓ @ylecun · May 27

Now you're acting as if you were my boss 😂😂😂

236

216

6.7K

518K



Backpropagation Applied to Handwritten Zip Code Recognition

Publisher: MIT Press

[Cite This](#)

[PDF](#)

Y. LeCun ; B. Boser ; J. S. Denker ; D. Henderson ; R. E. Howard ; W. Hubbard ; L. D. Jackel [All Authors](#)

2283

Cites in
Papers

48

Cites in
Patents

376

Full
Text Views



This document is outside of your subscription. For access:

- Contact the [IEEE Customer Center](#).
- Check to see if you have access via the [publisher's website](#).
- Purchase the document.

Abstract

[Authors](#)

[Citations](#)

[Metrics](#)

Abstract:

The ability of learning networks to generalize can be greatly enhanced by providing constraints from the task domain. This paper demonstrates how such constraints can be integrated into a backpropagation network through the architecture of the network. This approach has been successfully applied to the recognition of handwritten zip code digits provided by the U.S. Postal Service. A single network learns the entire recognition operation, going from the normalized image of the character to the final classification.

Published in: [Neural Computation](#) (Volume: 1 , Issue: 4, December 1989)

Page(s): 541 - 551

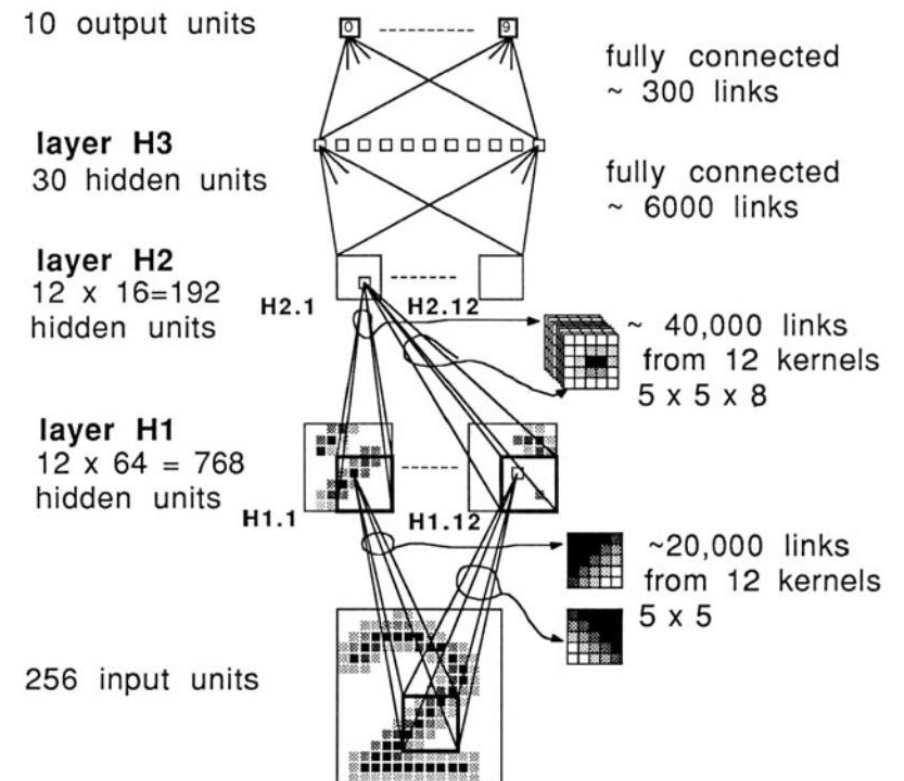
DOI: [10.1162/neco.1989.1.4.541](#)

Date of Publication: December 1989 [?](#)

Publisher: MIT Press

Print ISSN: 0899-7667

5.1 Comparison with Other Work. The first several stages of processing in our previous system (described in Denker *et al.* 1989) involved convolutions in which the coefficients had been laboriously hand designed. In the present system, the first two layers of the network are constrained to be convolutional, but the system automatically learns the coefficients that make up the kernels. This “constrained backpropagation” is the key to success of the present system: it not only builds in shift-invariance, but vastly reduces the entropy, the Vapnik–Chervonenkis dimensionality, and the number of free parameters, thereby proportionately reducing the amount of training data required to achieve a given level



Gradient-based learning applied to document recognition

Publisher: **IEEE**

[Cite This](#)

 **PDF**

Y. Lecun ; L. Bottou ; Y. Bengio ; P. Haffner [All Authors](#)

33102

Cites in
Papers

212

Cites in
Patents

102198

Full
Text Views



[Abstract](#)

[Authors](#)

[References](#)

[Citations](#)

[Keywords](#)

[Metrics](#)

Abstract:

Multilayer neural networks trained with the back-propagation algorithm constitute the best example of a successful gradient based learning technique. Given an appropriate network architecture, gradient-based learning algorithms can be used to synthesize a complex decision surface that can classify high-dimensional patterns, such as handwritten characters, with minimal preprocessing. This paper reviews various methods applied to handwritten character recognition and compares them on a standard handwritten digit recognition task. Convolutional neural networks, which are specifically designed to deal with the variability of 2D shapes, are shown to outperform all other techniques. Real-life document recognition systems are composed of multiple modules including field extraction, segmentation recognition, and language modeling. A new learning paradigm, called graph transformer networks (GTN), allows such multimodule systems to be trained globally using gradient-based methods so as to minimize an overall performance measure. Two systems for online handwriting recognition are described. Experiments demonstrate the advantage of global training, and the flexibility of graph transformer networks. A graph transformer network for reading a bank cheque is also described. It uses convolutional neural network character recognizers combined with global training techniques to provide record accuracy on business and personal cheques. It is deployed commercially and reads several million cheques per day.

Published in: [Proceedings of the IEEE](#) (Volume: 86 , Issue: 11 , November 1998)

I. INTRODUCTION

Over the last several years, machine learning techniques, particularly when applied to NN's, have played an increasingly important role in the design of pattern recognition systems. In fact, it could be argued that the availability of learning techniques has been a crucial factor in the recent success of pattern recognition applications such as continuous speech recognition and handwriting recognition.

The main message of this paper is that better pattern recognition systems can be built by relying more on automatic learning and less on hand-designed heuristics. This is made possible by recent progress in machine learning and computer technology. Using character recognition as a case study, we show that hand-crafted feature extraction can be advantageously replaced by carefully designed learning machines that operate directly on pixel images. Using document understanding as a case study, we show that the traditional way of building recognition systems by manually integrating individually designed modules can be replaced by a unified and well-principled design paradigm, called GTN's, which allows training all the modules to optimize a global performance criterion.

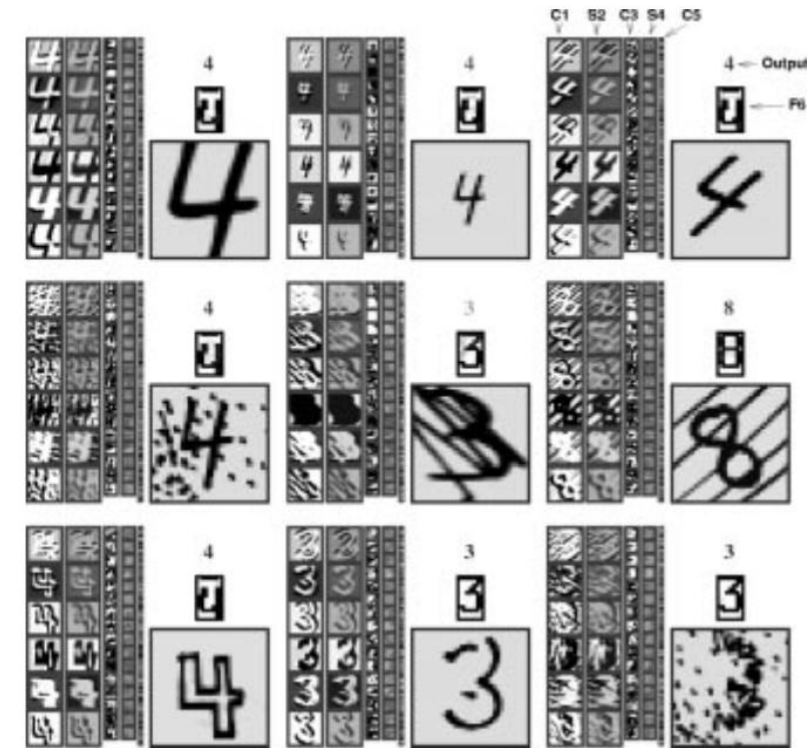
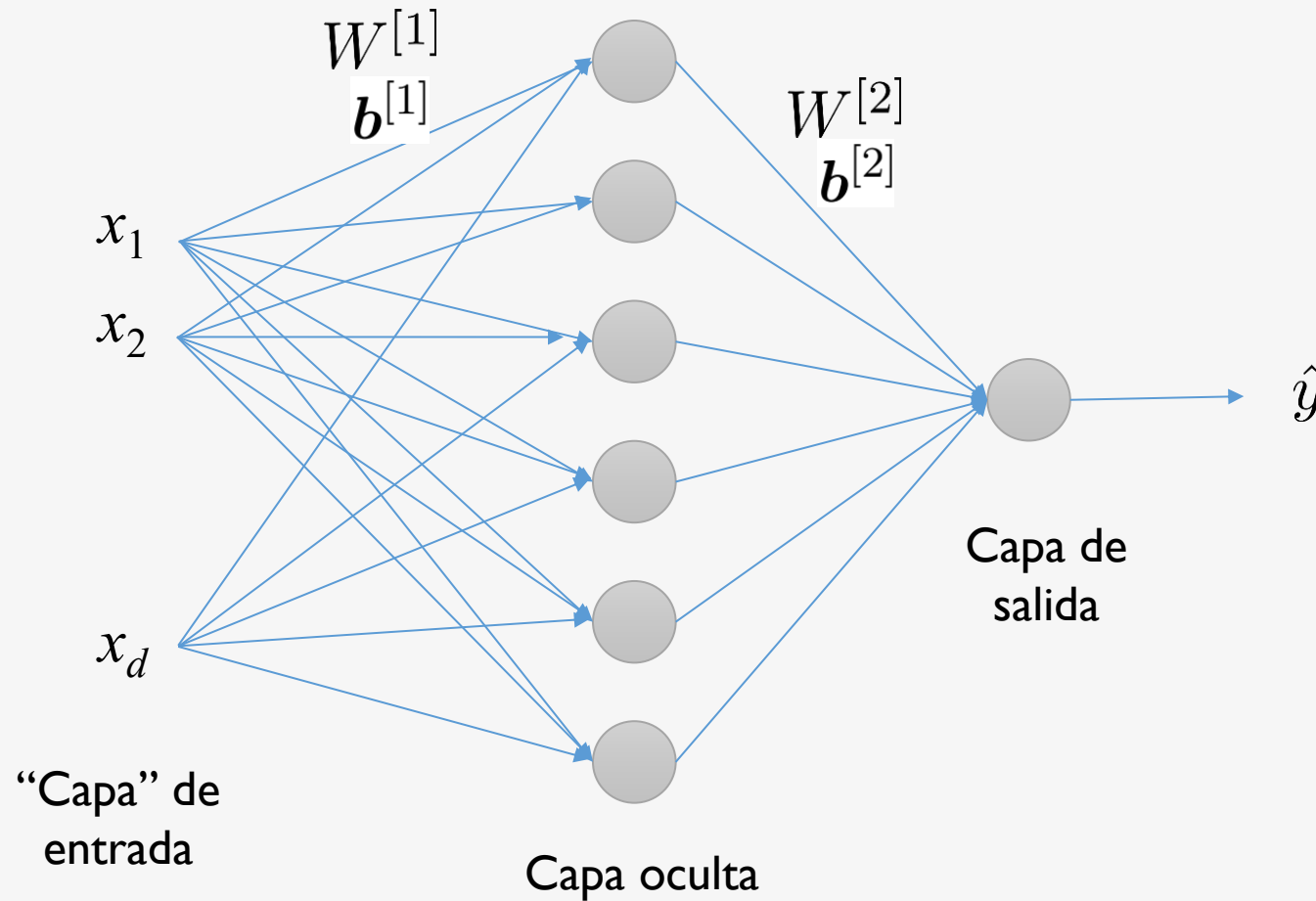


Fig. 13. Examples of unusual, distorted, and noisy characters correctly recognized by LeNet-5. The grey level of the output label represents the penalty (lighter for higher penalties).

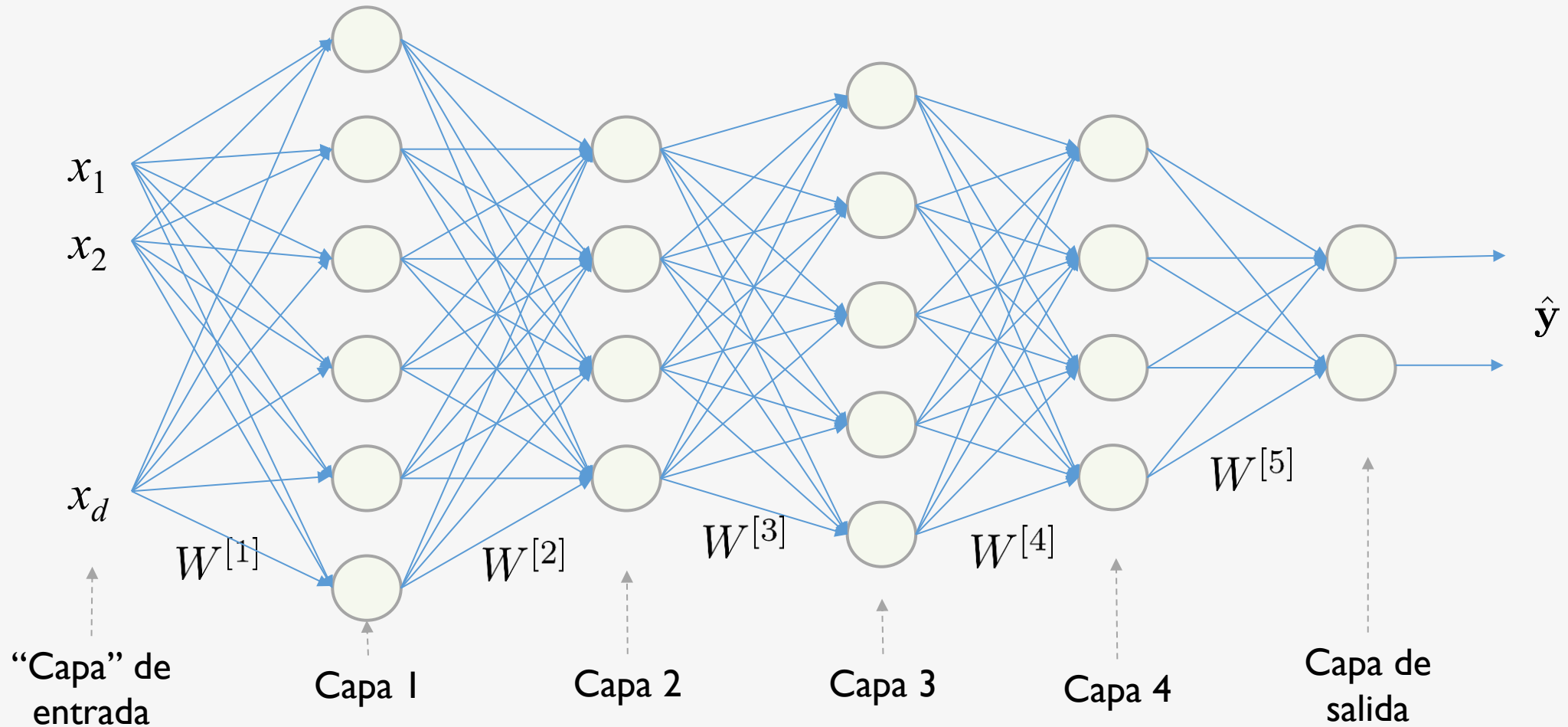
Repasando

Redes Neuronales Simples (pocas capas)



Redes Neuronales Profundas

- Llamadas: *Deep Neural Networks (DNN)*



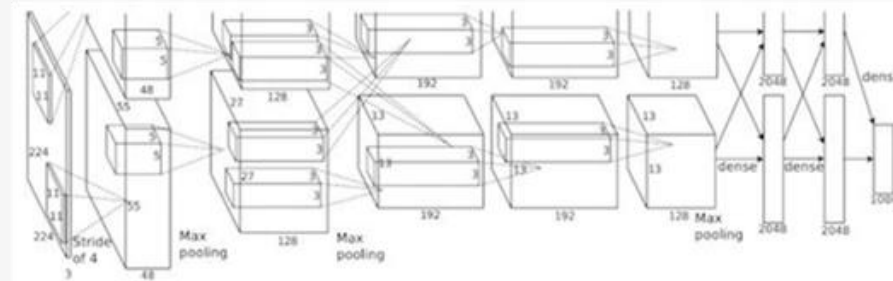
Aplicación en visión computacional

- Visión computacional “tradicional”
 - Se basa en extracción manual de atributos (features)
 - Los atributos se usan para el entrenamiento



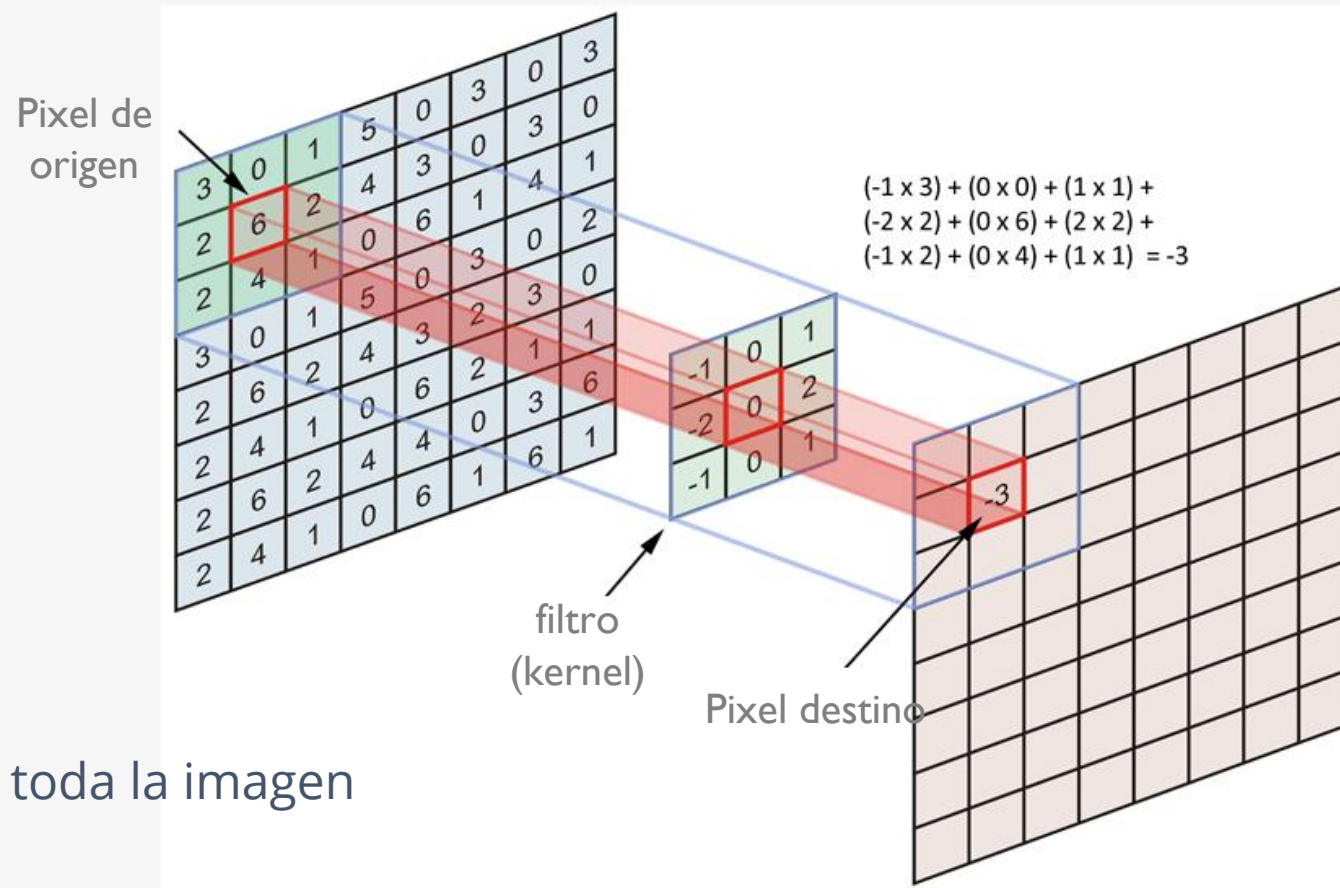
Entrenamiento basado
en atributos
(SVM, redes neuronales,
random forest, AdaBoost,
etc.)

- Visión computacional basada en CNN (“convolutional neural networks”)
 - Toda la imagen se usa para el entrenamiento



- Convolución consiste en:

- Sobreponer un filtro a la imagen, realizar las multiplicaciones término a término, y sumar el resultado



- El filtro “recorre” toda la imagen

Lectura recomendada

Applied Deep Learning - Part 4: Convolutional Neural Networks



Arden Dertat · Follow

Published in Towards Data Science · 23 min read · Nov 8, 2017



7.5K



47

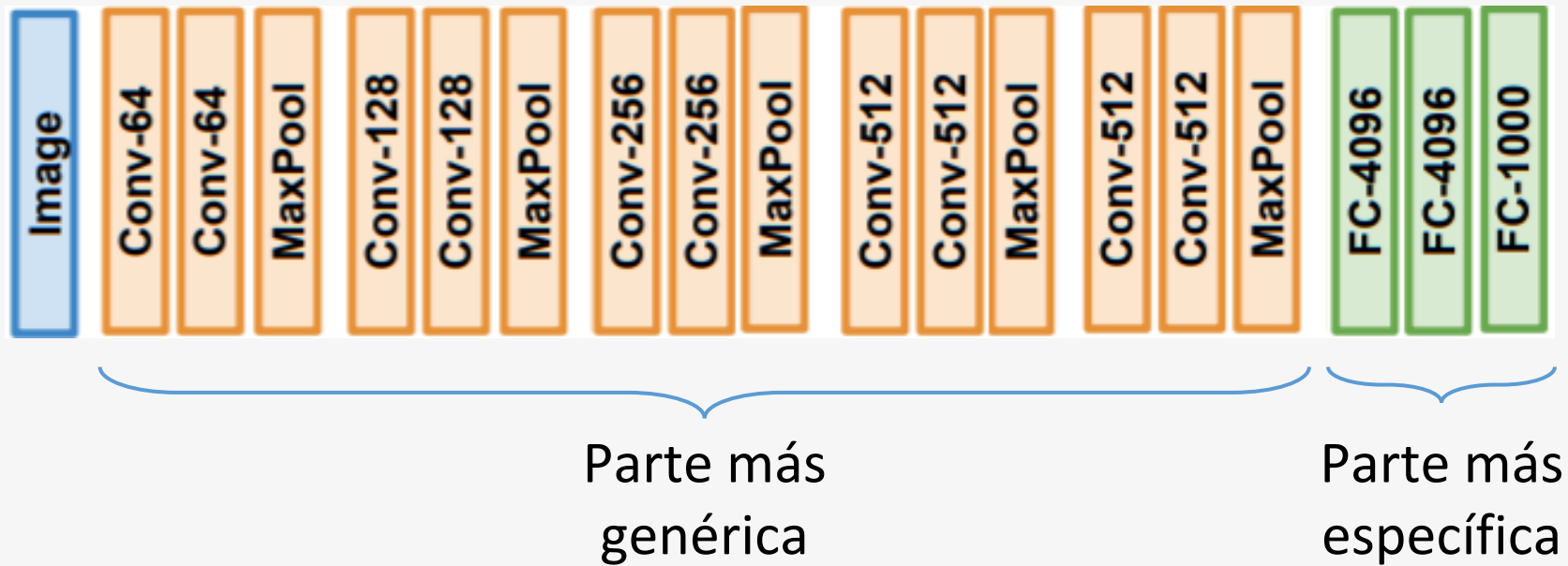


<https://towardsdatascience.com/applied-deep-learning-part-4-convolutional-neural-networks-584bc134c1e2>

Aplicaciones

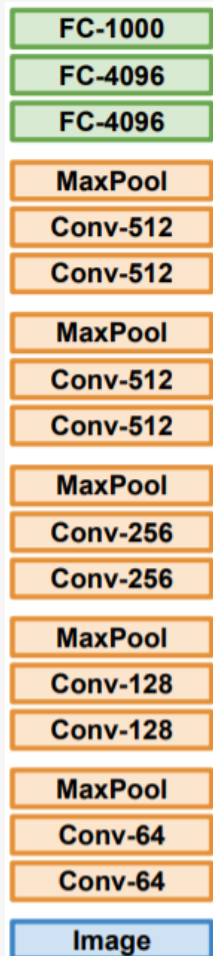
- Problema en CNNs:
 - Se requiere muchos datos para entrenar/usar CNNs
 - Requiere mucho tiempo de entrenamiento
- Alternativa
 - Aprovechar parte de alguna red ya entrenada (es común en la práctica)
 - Se conoce como "Transfer Learning"

Aplicaciones

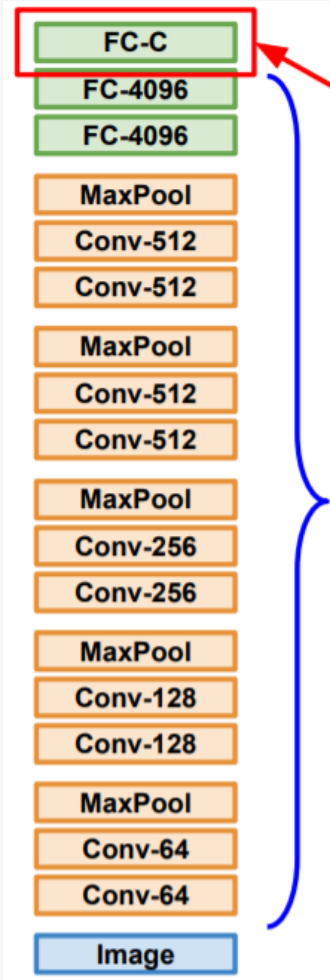


Aplicaciones

Red entrenada en
ImageNet



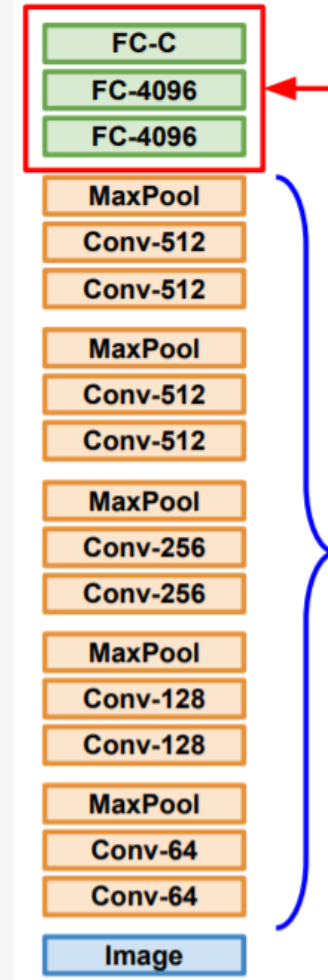
Conjunto de
entrenamiento pequeño



entrenar

mantener fijo (el original, no
entrenar)

Conjunto de
entrenamiento
más grande



entrenar

mantener fijo (el original, no
entrenar)

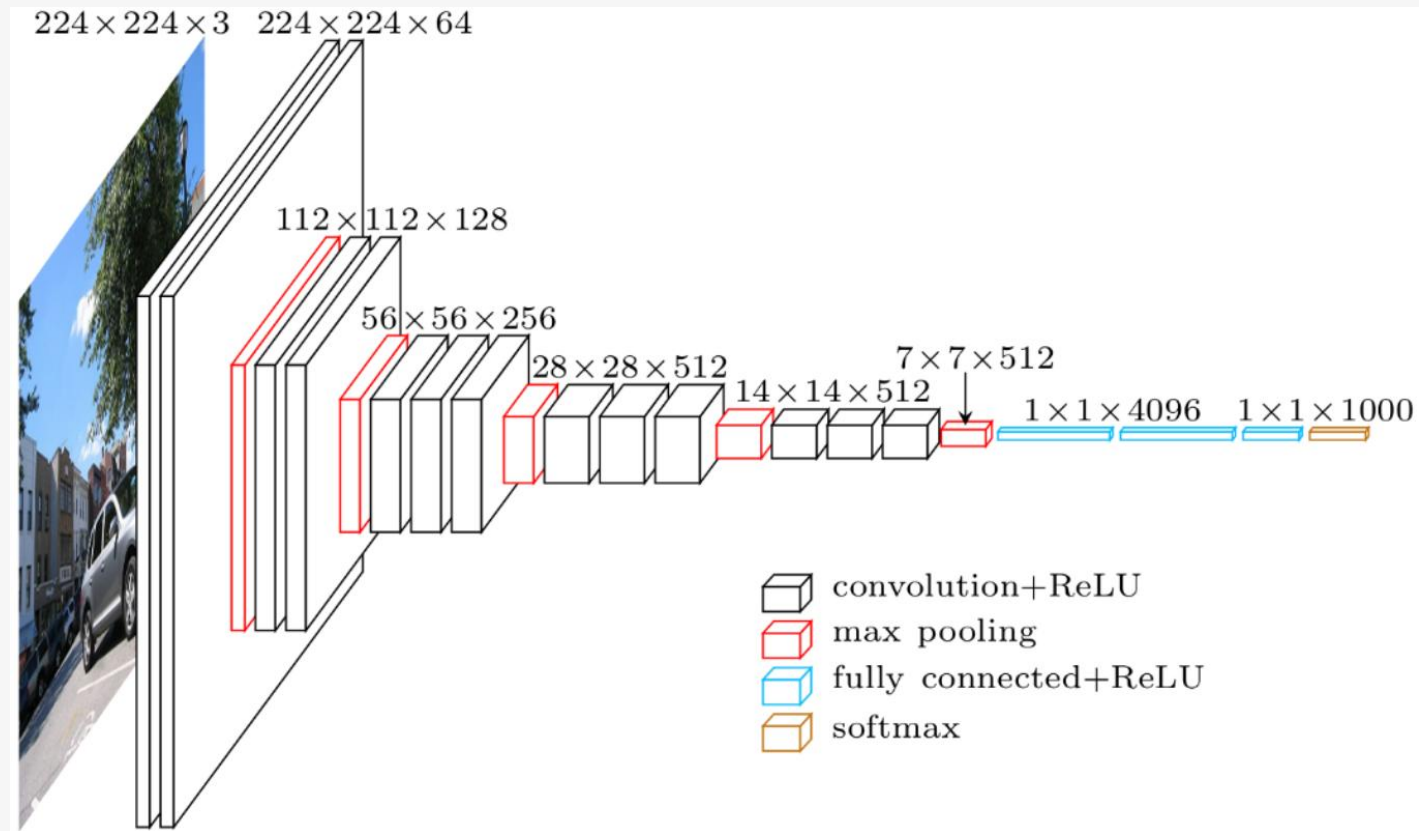
Aplicaciones

- Ideas para utilizar transfer learning
 - Según la cantidad de datos que se tenga
 - Según la similaridad o diferencia con el sistema pre-entrenado

	Dataset muy similar	Dataset muy diferente
Muy pocos datos	Hacer un “finetune” del clasificador de la capa final	Intentar “data augmentation” o recolectar más datos
Bastantes datos	Hacer un “finetune” de algunas capas	Hacer un “finetune” de un número más grande de capas

Ejemplos

VGG-16



K.Simonyan and A.Zisserman. "Very deep convolutional networks for large-scale image recognition." (2014).